



COSM: A Cooperative Scheduling Framework for Concurrent PIM and CPU Execution on Mobile Devices

Yilong Zhao^{*1,2} Fangxin Liu^{*1,2} Onur Mutlu³ Mingyu Gao^{4,2} Jian Liu⁵
Haibing Guan¹ Li Jiang^{†1,2,6}

¹Shanghai Jiao Tong University ²Shanghai Qi Zhi Institute ³ETH Zurich ⁴Tsinghua University
⁵Beihang University ⁶Huawei Technologies Co., Ltd.

The development of on-device large language models (LLMs) is driven by the need for privacy and fast response times. Energy-intensive data transfer on mobile devices makes Processing-in-Memory (PIM) an effective solution. Due to stringent DRAM cost constraints, limited physical footprint on circuit boards, and the interaction between applications and LLMs, it is imperative for the CPU and PIM to operate concurrently within a shared memory space. However, challenges such as bank conflicts and bus congestion can arise, potentially diminishing the performance and energy benefits of PIM.

To address this challenge, we introduce COSM, a cooperative scheduling framework designed to facilitate the concurrent operation of PIM and CPU tasks on mobile platforms. Our key innovations include: 1) a low-interference PIM control interface that generates the maximum number of PIM commands without disrupting CPU memory accesses; 2) an idleness-aware scheduling method that integrates PIM commands into available idle time windows within the CPU's access sequence. COSM not only hides PIM execution latency from the CPU, but also overlaps PIM execution with data transfer. Experiments on concurrent execution of LLMs and mobile workloads, including mobile applications and compute-intensive kernels, demonstrate that COSM improves PIM throughput by up to 2.8× compared to the baseline scheduling method with less than 2.0% CPU performance loss.

Index Terms— processing-in-memory (PIM), memory scheduling, mobile devices, LLM inference, memory interference

1. Introduction

AI advancements are moving large language models (LLMs) from the cloud to edge devices (e.g., mobile phones and PCs), enhancing privacy by keeping data local and enabling millisecond latency for interactive apps like voice assistants [1–3], real-time translation [4,5], real-time video understanding [6], and video editing [7]. Industry trends show that companies such as Apple [8], Huawei [9], Qualcomm [10], Samsung [11], and Vivo [12] have integrated AI on the device with models of 1B–3B parameters, highlighting “localized intelligence” as a key feature of future smart devices. Thus, efficiently running LLMs on resource-limited mobile devices has become a major challenge for both academia and industry.

Current on-device deployment strategies using Neural Processing Units (NPUs) have notable limitations. Despite high compute throughput, current devices’ and NPUs’ processor-

centric architecture spends over 60% of its total energy consumption on data movement during LLM inference, causing thermal challenges and shortening battery life [13]. For instance, a 7B model on a mobile NPU can draw over 450 mA current [14]. Moreover, the limited LPDDR5X memory bandwidth (typically <80 GB/s) also leads to latency fluctuations during long-context generation [13].

Processing-in-Memory (PIM) directly integrates computation units (PIM units) near memory banks in a DRAM chip to overcome limited memory bandwidth [15–35]. This innovation enhances internal bandwidth (i.e., the bandwidth from the memory banks and row buffers to either I/O circuitry or PIM units) by processing data locally, thereby bypassing the bottleneck of external bandwidth (i.e., the bandwidth across the memory bus). Samsung’s LPDDR5-PIM prototype showed a 70% reduction in power consumption and 102.4 GB/s memory bandwidth in mobile settings [36]. Thus, PIM is emerging as a promising and effective architectural paradigm to overcome the energy and memory bottlenecks of mobile AI hardware [36].

Some DRAM-PIM designs, like UPMEM’s DDR4-PIM [15], enforce static partitioning between CPU and PIM memory spaces, which incurs substantial memory reservation overhead for LLMs and significant data movement between CPU and PIM units. Shared-memory PIM architectures, where the CPU and PIM units physically share the same memory space rather than operating in isolated memory spaces, address this problem by using OS-managed logical isolation instead but they introduce new challenges in memory management [18,37] and bandwidth scheduling [38–41]. While recent works aim to tackle these issues, the industry has not yet deeply explored this direction. We observe complementary bandwidth usage: *CPUs use high external DRAM bandwidth but low internal DRAM bandwidth, opposite to PIM units.* This allows us to leverage idle internal bandwidth of CPU workloads for PIM tasks, enhancing memory efficiency and offering dual benefits for mobile devices: reduced overhead due to dynamic sharings of banks between CPU and PIM units and lower power consumption due to less data movement.

The shared memory design aims to harvest idle DRAM bandwidth for PIM workloads without significantly impacting CPU performance. Current PIM scheduling frameworks are hindered by memory scheduling and PIM control interface constraints. Memory scheduling strategies have trade-offs: CPU-first scheduling [39] maintains CPU latency but underutilizes internal bandwidth, while row-hit-aware scheduling [38,40] optimizes bandwidth utilization and PIM performance but de-

^{*} Yilong Zhao and Fangxin Liu contribute equally to this work. [†] Li Jiang is the corresponding author.

grades CPU performance. Both strategies struggle to balance PIM and CPU performance. Additionally, PIM control limitations further exacerbate the scheduling difficulty. PIM architectures with fine-grained commands minimize CPU blocking, but can saturate the command bus, reducing PIM throughput [16, 42]. In addition, CPU-initiated data transfers in PIM workloads for staging inputs and collecting results, referred to as CPU-mediated transfer, share data paths with CPU access, causing contention and degrading CPU performance.

In this work, we present COSM, a novel cooperative scheduling framework for concurrent PIM/CPU execution designed to balance CPU and PIM performance trade-offs. We first propose a *low-interference PIM control interface* that includes two key mechanisms: (1) a *preemptable PIM execution command* to mitigate command bus contention while ensuring responsive CPU access, and (2) a *bandwidth-decoupled data transfer command* to prevent PIM data transfer from stalling CPU accesses. Built on this interface, we propose an *idleness-aware scheduling strategy* in the memory controller. Specifically, by monitoring the CPU access queue, the controller identifies idle time windows on both the memory bus and banks. It then schedules PIM commands, including PIM execution and data transfer, within these idle time windows, to fully exploit idle bandwidth while minimizing interference with CPU memory accesses. This strategy enhances resource utilization while incurring minimal impact on CPU performance.

We make the following contributions:

- We provide key observations that identify critical factors affecting the performance of both CPU and PIM workloads, offering design insights for PIM systems. While CPU workloads are sensitive to memory access latency, existing PIM designs cause significant interference during both PIM execution and CPU-mediated data transfers. This is a conflict rooted in both the PIM control interface and memory scheduling strategy. Particularly, CPU-mediated data transfers within PIM workloads incur substantial performance degradation for CPU tasks.
- We propose COSM, a new cooperative scheduling framework for concurrent PIM/CPU execution on mobile devices. At the interface level, COSM introduces a low-interference PIM control interface tailored to our observations of CPU and PIM workload conflicts. Building on this, our scheduling policy precisely coordinates PIM command dispatch to exploit CPU idle time windows of CPU access, keeping CPU access latency low while maximizing PIM performance.
- Our comprehensive experiments show that COSM improves LLM throughput on PIM by 2.6× with less than 2.2% performance degradation on concurrent CPU workloads.

2. Background

2.1. Hierarchical Architecture of DRAM

In modern DRAM, the latency of control commands (e.g., opening a row in LPDDR5 [43] requires $tRP + tRCD \approx 30$ DRAM clock cycles) is much longer than the duration of the burst length ($tBL = 8$ DRAM clock cycles), where tRP , $tRCD$, and tBL refer to row precharge time, the row address strobe to column address strobe delay, and the burst length [44, 45].

To mitigate this overhead, DRAM adopts a hierarchical architecture. Multiple banks in a rank can operate independently, and row operations can overlap with each other across banks that share a common memory bus.

As DRAM density increases with increasing bank counts, the overhead of row opening emerges as a fundamental scaling bottleneck. Although more banks offer higher theoretical parallelism, fixed command bandwidth severely restricts maximum internal bandwidth utilization. For example, in a typical 2-rank LPDDR5 per-channel mobile phone setup, each rank has 32 banks [36] ($\#bank = 32$). For a workload with a row hit rate R_h , the upper bound under saturated external bandwidth (i.e., assuming 100% external bandwidth utilization) is:

$$Util_i = \frac{tBL + (tRP + tRCD) \cdot (1 - R_h)}{\#bank \cdot tBL} \quad (1)$$

The $\#bank$ term in the denominator illustrates that the shared command bus serializes bank accesses, causing the denominator to far outweigh the numerator. This reveals a fundamental problem: *even with completely random access that leads to zero hit rate ($R_h = 0$), the utilization of internal bandwidth cannot exceed 15%*. Real-world workloads typically achieve far lower utilization, as demonstrated in Fig. 1.

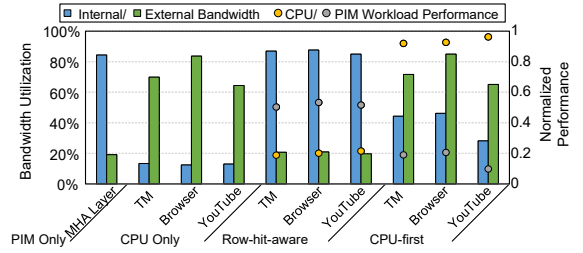


Fig. 1: Internal/external DRAM bandwidth utilization of CPU and PIM workloads, and CPU/PIM workload performance under different scheduling strategies when concurrently executing on physically shared memory space. (Note: TM stands for TencentMeeting.)

2.2. DRAM-based PIM

Unlike traditional heterogeneous systems (e.g., CPU-GPU systems) where all compute units share a unified DRAM controller interface, PIM units are spatially distributed across DRAM banks and access memory through dedicated intra-bank pathways. This fundamental architectural distinction requires specialized interfaces for PIM systems. Current implementations predominantly adopt two types of PIM interfaces:

2.2.1. Two-Host Design. A PIM unit functions independently from the CPU, with its own instruction sequencer and local DRAM access [15, 33, 34, 46–49]. During PIM operations, CPU access to these banks is blocked to prevent DRAM state corruption. Completion of PIM tasks is detected through polling, which checks status registers. After execution, the memory controller must resynchronize DRAM for CPU access, causing significant switching overhead between CPU and PIM access.

2.2.2. Single-Host Design. PIM units use extended DRAM commands for precise control, optimized for specific PIM workloads due to limited command encoding space [16, 30, 38, 39, 42, 50, 51]. Recent advances include translation tables that map high-level operations to these commands, en-

hancing flexibility. This design, unlike the two-host model, keeps the memory controller fully aware of changes in DRAM state during PIM operations. Its centralized scheduling allows fine-grained interleaving of CPU and PIM commands without extra synchronization, supporting efficient concurrency by removing conservative timing or polling requirements.

3. Key Observations and Design Implications

We analyze CPU and PIM workload interactions in a shared-memory CPU/PIM hybrid system with concurrent execution. We present three key observations affecting performance and analyze concurrent execution techniques. These insights drive our interface and scheduling co-design in Section 5 and 6.

3.1. Effect of Latency Interference on Memory Access

To characterize the impact of memory-side contention, we conduct a sensitivity study on CPU performance in response to increased memory latency. We simulate PIM-induced access delays by injecting additional CPU read latency on a CPU-only system across three real-world applications and a SPEC 2017 benchmark. As shown in Figure 2(a), a 16-cycle latency increase reduces CPU performance by more than 5%. As latency increases, performance significantly degrades: e.g., a 128-cycle latency reduces CPU performance by more than 40% for some workloads. This confirms that CPU workload performance is sensitive to memory access latency.

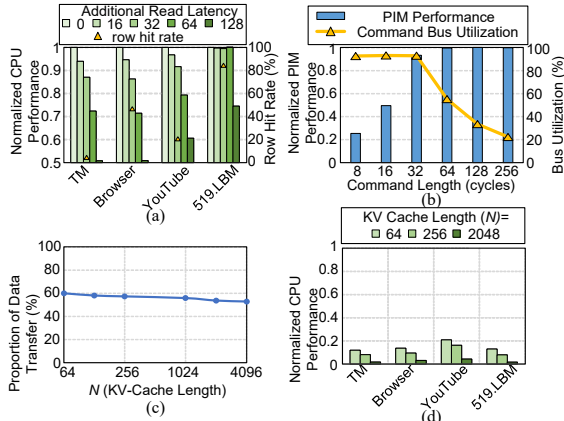


Fig. 2: (a) CPU workload performance under injected read latency. (b) PIM workload performance and command bus occupation across command lengths. Performance is normalized to the peak performance under an unsaturated command bus (command length ≥ 64). (c) Proportion of CPU-mediated transfer in an attention layer inference of DeepSeek-R1-1.5B. (d) CPU workload performance when concurrently executing with CPU-mediated PIM data transfer.

The results of this study offer two key guidelines for the formulation of memory scheduling strategies in the concurrent execution of CPU-PIM hybrid systems. First, the system must enable fast preemption of PIM operations to reduce CPU request latency. This requires interrupting PIM operations and minimizing the switching overhead between CPU and PIM commands. A single-host design with fine-grained PIM command control is preferred, as a two-host design introduces significant switching latency, and coarse-grained PIM commands hinder timely CPU access. Second, memory scheduling

should give precedence to CPU memory accesses and restrict PIM operations to periods when the memory is idle, thereby guaranteeing minimal disruption. Collectively, these guidelines require a closely integrated design of the PIM execution interface alongside the memory scheduler.

3.2. Effect of PIM Command Granularity on Performance

We study how the granularity of PIM execution commands affects PIM performance, defining the “command length” as the number of cycles PIM units can execute autonomously per command. Figure 2(b) shows that longer command lengths improve PIM performance in a single-host system with 2 LPDDR5 ranks per channel and 32 banks in total. A command with a length of ≥ 128 keeps the command bandwidth below 40%, while a length of ≥ 64 is needed for full bank-level parallelism across 32 banks (taking into account the opening overheads of rows). Shorter command lengths cause command bus congestion and external DRAM bandwidth underutilization.

This finding is in tension with the observation in Section 3.1: longer command lengths boost PIM performance, but shorter ones are crucial to minimize CPU memory latency. To eliminate this tension in existing fixed-length command architectures, we propose *preemptable PIM execution commands* in the COSM framework to balance extended command benefits with CPU access needs.

3.3. Interference from CPU-mediated PIM Data Transfer

Although PIM workloads primarily leverage internal DRAM bandwidth between PIM units and their local banks, the host CPU still needs to transfer input data to PIM units and collect results from PIM units. These necessary CPU-mediated transfers create memory contention for co-executing CPU workloads that share the same DRAM devices.

To quantify the impact of this bandwidth contention, we first measure the proportion of CPU-mediated transfers during an attention layer inference of a DeepSeek-R1-1.5B model [52] in the decoding phase on the same system as Section 3.2, as shown in Fig. 2(c). The command length of each PIM execution is set to 128 cycles. When the sequence length dimension N of Key-Value (KV) cache is 64, CPU-mediated transfers account for over 60% of the overall inference time. As N increases during inference, this proportion gradually decreases to around 50% when $N=4k$, demonstrating that CPU-mediated transfers still constitute a substantial fraction of the overall execution time.

Fig. 2(d) further demonstrates the interference caused by CPU-mediated transfers on concurrent CPU workloads. Both CPU-mediated transfers of PIM workload and conventional CPU memory requests in CPU workload share the same memory request queue, which is scheduled using the classic First-Ready, First-Come First-Serve (FR-FCFS) policy [53,54]. We observe that CPU workload performance degrades by more than 80% when concurrent with CPU-mediated transfers. This severe slowdown arises because CPU-mediated transfers exhibit bursty access patterns with high row-buffer locality. Under the FR-FCFS scheduler, such bursts monopolize the memory controller (as described in [55–57]) by repeatedly hitting open

rows, thereby starving CPU workloads, whose requests often require costly row conflicts.

These results reveal a critical insight: the effect of the observed 80% CPU performance degradation during the 50% data transfer can translate into a 40% reduction in overall CPU performance. This disproportionate impact shows that CPU-mediated transfers must be treated as first-class citizens in memory scheduling in concurrent CPU and PIM access scenarios. Consequently, CPU-mediated transfers must be governed by the same scheduling discipline as PIM units' execution, i.e., scheduled during idle time windows. However, implementing the CPU-mediated transfers requires simultaneous availability of *both external and internal* idle time windows, making it difficult to fully utilize memory idle time windows, limiting PIM throughput. To overcome this, COSM introduces *bandwidth-decoupled CPU-mediated transfer commands* that decouple the usage of external and internal bandwidth, eliminating the need for simultaneous external and internal idle time window availability.

3.4. Concurrent CPU-PIM Execution Methods

Based on the observations above, we analyze several techniques for concurrent execution of PIM and CPU on shared memory banks. Fig. 3(a) depicts CPU memory request processing under FR-FCFS, showing memory bus and bank occupancy along with per-bank queue lengths. Fig. 3(b)-(d) compare three concurrency techniques under the same CPU workload. Each PIM workload consists of one burst write of CPU-mediated transfer (co-scheduled with normal CPU access under FR-FCFS) followed by multiple execution commands.

3.4.1. All-bank PIM Command Interface. The *all-bank* command mitigates command bandwidth bottlenecks by invoking all PIM units with one PIM command [42]. As shown in Fig. 3(b), this method integrates a PIM command amidst CPU memory requests. However, it causes bandwidth underutilization: all-bank command execution leaves no bank available for CPU accesses, making external bandwidth idle (①), and PIM commands must wait for all banks to become available, causing latency (②).

3.4.2. CPU-first Scheduling. Chopim [39] introduces a memory scheduling algorithm based on the idea that CPU workloads are sensitive to memory latency. It blocks PIM commands once the memory queue is detected as not empty, as shown in Fig. 3(c). While PIM commands minimally impact CPU latency, bandwidth utilization remains suboptimal. When a request targets Row 1 of Bank 1, the scheduler immediately stops issuing PIM commands. However, due to bus contention, there is a latency gap between row activation command (ACT) and data access, leading to wasted internal bandwidth (③).

3.4.3. Row-hit-aware Scheduling. AsyncDIMM [38] and F3FS [40] suggest that the memory controller switches between the PIM command queue and the CPU request queue when a row-close command (PRE) is executed, benefiting memory-intensive tasks with high row hit rates by balancing the CPU and PIM scheduling. However, with this policy, CPU performance suffers in compute-intensive tasks with random access. As shown in Fig. 3(d), closing row 1 in bank 0 forces the memory controller to switch to the PIM command queue, causing

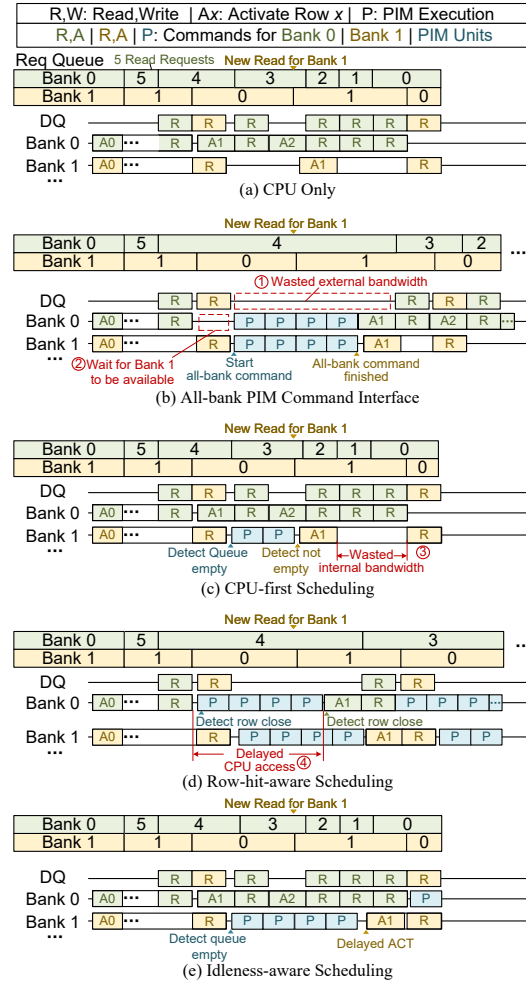


Fig. 3: (a) An example of FR-FCFS scheduling for a CPU-only workload and (b-e) methods for concurrent PIM/CPU execution. To simplify the diagram, precharge is combined with activation, and row activations prior to PIM executions are not shown.

CPU access delays and higher memory latency (④).

As such, prior scheduling methods cannot both provide low CPU memory latency and allow PIM workloads to fully utilize the temporary idle periods across banks on mobile devices. Furthermore, prior scheduling methods lack a dedicated scheduling policy for CPU-mediated data transfers, which interferes with CPU workloads as observed in Section 3.3.

4. Architecture Overview

We introduce a new scheduling framework in the memory controller to enhance the use of PIM workload bandwidth without affecting CPU memory latency. Our approach opportunistically schedules PIM operations during CPU request idle times, ensuring minimal CPU interference. COSM employs a dual-path SW/HW co-design. First, we extend the DRAM command set to enable a low-interference PIM control interface (see Section 5), forcing PIM tasks to pause for high-priority CPU requests, and create dedicated commands for CPU-mediated data transfers that decouple internal and external bandwidth

usage. Second, we develop an idleness-aware scheduling policy (see Section 6) that inserts PIM commands during idle time windows of CPU access while maintaining CPU latency guarantees, by enhancing the memory controller hardware.

Figure 4 shows the architecture of the COSM memory controller. We extend the software layer on the CPU side to accommodate a low-interference memory interface. A PIM Execution Engine (PEE) in PIM units manages preemptable command execution and prevents command bus saturation. We use an SRAM buffer next to each DRAM bank to buffer both the operands for PIM execution and the data staged by the decoupled-bandwidth CPU-mediated transfer commands. facilitate both CPU-mediated transfers and PIM execution. Within the memory controller, we introduce two dedicated PIM queues: the PIM Execution Queue (PEQ), which buffers commands for PIM computations (e.g., `PIM_Exec(L)`, `PIM_Exec(S)`), and the PIM Read/Write Queue (PRWQ), which manages CPU-mediated transfer commands (e.g., `PIM_Ld/StB`, `PIM_Rd/WrB`). The PIM scheduler is responsible for selecting candidate commands from these queues, while the idle time window estimator (IWE) analyzes patterns in the CPU access queue to predict forthcoming bank- and bus-level idle time windows in between CPU accesses. The Command Arbiter consolidates and chooses from three command sources: 1) the CPU memory access candidate from the traditional FR-FCFS scheduler, 2) the PIM candidate from the PIM scheduler, and 3) pause commands to pause PIM execution (`PIM_Pause`) generated by the Command Arbiter itself. The Command Arbiter employs a strict priority policy to decide the final sequence for issuing DRAM commands (Section 6.2).

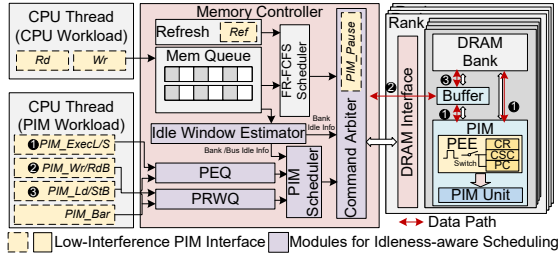


Fig. 4: COSM’s memory controller architecture and memory interface

The controller operates in a four-stage pipeline. First, requests are queued by command type. Second, the IWE examines access queues to predict idle time windows, sending its predictions to the Command Arbiter and PIM scheduler. Third, the FR-FCFS and PIM schedulers each select commands based on their prioritization policies. Finally, the Command Arbiter makes decision: If a PIM-induced stall is predicted for a CPU memory request, the Command Arbiter issues a `PIM_Pause` to prevent delaying the CPU accesses. Otherwise, priority is granted to the FR-FCFS scheduler (i.e., serving CPU access and refresh requests). If no such requests are available, a PIM command is scheduled.

5. Low-Interference PIM Control Interface

COSM introduces a refined PIM control interface that explicitly provide two separate new features: (1) preemptable PIM execution for compute phases, and (2) bandwidth-decoupled

commands for CPU-mediated data movement. This enables fine-grained scheduling control, allowing PIM operations to yield instantly to CPU requests and exploit fragmented idle time windows for PIM commands without compromising data staging efficiency.

5.1. Preemptable PIM Execution commands

The proposed preemptable PIM control interface extends standard DRAM commands with two critical additions: PIM execution command family (`PIM_Exec`) that allow PIM units to automatically execute computations continuously for an extended period without requiring additional trigger commands, and a PIM pause command (`PIM_Pause`) that enables preemption by halting PIM execution. Compared to the fixed-length command design discussed in Sections 3.1 and 3.2, the preemptable commands enable immediate reaction to incoming CPU access while avoiding command bus saturation that would otherwise degrade PIM performance. The scope of all the PIM command are at the bank level. We employ a hardware-command co-design approach to achieve preemptable computation. Managed by the memory controller, this interface allows fine-grained execution of PIM workloads while enabling low CPU memory access latency.

PIM Execution Commands. In our setup, the execution commands `PIM_Exec` for PIM are tailored for LLM operations such as MAC and softmax. Fig. 5 (a) illustrates the execution of a PIM command and the register states of the PEE. If a command with start column 64 is issued at $clk1$, the PEE module of the PIM unit switches from state ① to ② by setting the Column State Counter (CSC) with the start address, recording the command type in the Command Register (CR), and setting the PIM Counter (PC) as 0. For every t_{CCD} (column-to-column delay), the PEE autonomously increments the Column State Counter and PIM Counter and issues commands in Command Register using the Column State Counter as the column address (③). The Command Arbiter can always synchronize the PIM Counter state by inferring the PIM Counter : $PC_{inf} = (clk - clk1) / t_{CCD}$, where clk is the current clock cycle. This procedure halts after n_{PTL} cycles (i.e., PIM execution command length predefined by the DRAM configuration register) by comparing PIM Counter value with the n_{PTL} / t_{CCD} value if there is no CPU memory access, and PEE returns to its default state (④). This reduces the usage of the command bus by n_{PTL} / t_{CCD} times compared to fine-grained commands (see Section 3.2). In the setup shown in Fig. 2 (b) (2 ranks $\times$ 16 banks), n_{PTL} needs to be at least 64 cycles for more flexibility, with lower values leading to command bus contention.

PIM Pause Command. The `PIM_Pause` command ensures timely CPU request handling by pausing PIM execution. Fig. 5 (b) shows a CPU memory request interrupting PIM execution when the CPU reads from a bank still processing column 65 at time $clk2 = clk1 + t_{CCD} + 1$. The memory controller’s Command Arbiter sends `PIM_Pause` to the bank. Upon receiving it, the PEE completes the current column operation in up to t_{CCD} cycles, freezes the Column State Counter (by opening the Switch of PEE in Fig. 4), and releases the bus control (from ⑤ to ⑥). The Command Arbiter can directly infer the PIM Counter state by calculating the interval between

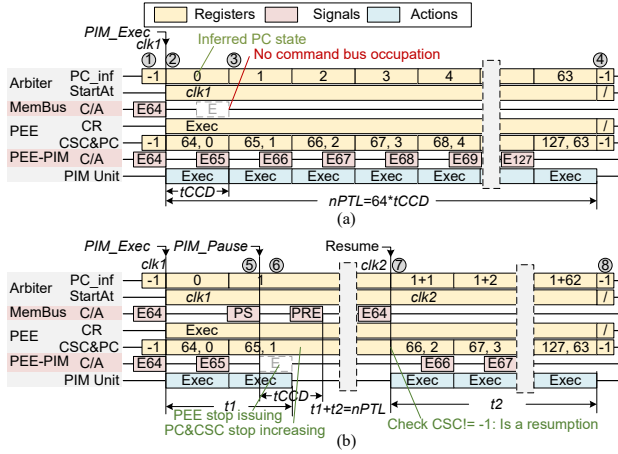


Fig. 5: Timing diagram of PIM unit, memory bus, Command Arbiter, and PEE register states of preemptible PIM execution command when (a) no CPU memory access is present and (b) PIM_Pause is sent by Command Arbiter when processing the second column. En : PIM execution on column n , PS: PIM pause command.

$clk1$ and the end of the current execution ($t1 = 2t_{CCD}$). After CPU access, the Command Arbiter reissues the origin PIM execution command, resuming from the Column State Counter-stored column address (⑦). Knowing that columns 64-65 are completed, the Command Arbiter determines the remaining execution time as $t2 = nPTL - 2t_{CCD} = 62t_{CCD}$ (⑧), synchronizing with the PIM Counter without extra signaling and minimizing timing constraints.

Timing Constraints. Table 1 provides the timing constraints for preemptible PIM commands. Similar to DRAM read/write commands, a PIM execution command can only be issued after t_{RCD} after row activation. The PIM_Pause follows at least t_{CCD} cycles after PIM execution command to ensure PIM unit column access completion. Post PIM_Pause, if the execution command is load-only (PIM_Exec(Ld)), a PRE can be issued after t_{RTP} cycles. For execution commands with data stores (PIM_Exec(St)), PRE must wait for data stabilization, requiring t_{WR} . This longer latency necessitates careful design to maximize intermediate result reuse and minimize store operations.

Table 1: Timing Constraint of PIM Commands

Scope	Previous	Next	Min. delay	Conflict
Bank	ACT	PIM_Exec PIM_LdBuf/StBuf	t_{RCD}	DRAM Array
	PIM_Exec PIM_Ld/StBuf	PIM_Pause	t_{CCD}	
	PIM_Pause (for PIM_Exec(Ld))	PRE	t_{RTP}	
	PIM_Pause (for PIM_Exec(St))	PRE	$t_{CCD} + t_{WR}$	
	PIM_RdBuf/WrBuf PIM_LdBuf/StBuf	PIM_RdBuf/WrBuf PIM_LdBuf/StBuf	t_{BL}	PIM Buffer
Channel	PIM_RdBuf/WrBuf Read/Write	PIM_RdBuf/WrBuf Read/Write	t_{BL}	Memory Bus

Semantic Guarantees of PIM_Pause. The correctness of the PIM pause mechanism relies on strict adherence to DRAM timing constraints and three fundamental properties. First, it guarantees *column atomicity* by pausing only at column boundaries (after integer multiples of t_{CCD}), ensuring deterministic

progress tracking for both the PIM unit and the memory controller. Second, it preserves all intermediate data in PIM buffers and architectural states during suspension, preventing context corruption. Third, it correctly restores PIM state by reactivating the original row and reissuing the paused command; the PIM unit then leverages frozen PIM Counter to resume execution precisely at the pause point. Collectively, these guarantees ensure that preempted CPU accesses and DRAM refreshes do not compromise the correctness of PIM execution.

5.2. Bandwidth-Decoupled CPU-mediated PIM Data Transfer

Conventional PIM data transfers are based on standard DRAM read/write commands, which could require a long sequence of row activation, column access, and external data transfer. Moreover, the CPU-mediated transfer must occur continuously, locking the channel for the entire duration of the transfer.

We introduce a mechanism that decouples the usage of external bandwidth from that of internal bandwidth during CPU-mediated data transfer. As depicted in Fig. 6, the memory interface employs two-phase reads and writes via four commands: PIM_RdBuf and PIM_WrBuf for *external transfers* between the memory controller and the PIM buffer through the memory bus, and PIM_LdBuf and PIM_StBuf for *internal transfers* between the buffer and DRAM banks. This division allows independent scheduling of requests, minimizing CPU memory access interference by using bus idle time windows for external transfers and exploiting bank idle time window for internal transfers. It enables the idleness-aware scheduling policy detailed in Section 6.2.

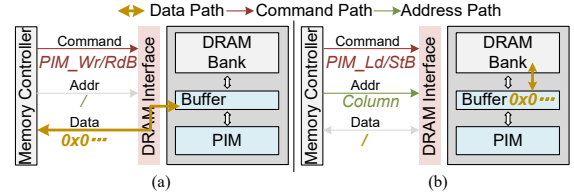


Fig. 6: The data, command, and address path of (a) PIM_WrBuf/RdBuf and (b) PIM_LdBuf/StBuf

The PIM_LdBuf and PIM_StBuf commands are designed to mirror the memory access behavior of PIM_Exec(Ld) and PIM_Exec(St): they exclusively access the DRAM bank using internal bandwidth, transferring data from/to the bank to/from the buffer. and do not consume external bandwidth. Consequently, they inherit the same execution model: preemptible operation with a command length of $nPTL$. This design choice enables unified scheduling logic in all PIM-initiated bank activities.

Additional PIM Buffer Requirement. To support our mechanism, we add an extra segment to the PIM buffer whose capacity is commensurate with the maximum amount of data transferred by a single PIM_LdBuf or PIM_StBuf command of length $nPTL$. Since internal bandwidth is frequently available during ample idle time windows, the scheduler can always complete these short internal transfers before the next external transfer command arrives. Thus, a small per-bank buffer suffices to avoid stalling, as validated in our experiment (Section 8.5).

Timing Constraints. Table 1 provides the timing constraints for the new commands. PIM_RdBuf and PIM_WrBuf commands only use the external memory channel bandwidth, requiring a t_{BL} delay after standard Read/Write commands on the same channel. These two commands, along with PIM_LdBuf/PIM_StBuf, need the PIM buffer to be ready before execution, adding a t_{BL} delay when issued to the same bank. Additionally, PIM_LdBuf and PIM_StBuf access the DRAM bank like PIM_Exec(Ld) and PIM_Exec(St), following the same timing as DRAM commands like ACT, PRE, Read, and Write.

Memory Ordering and Buffer Consistency. The bandwidth-decoupled transfer mechanism guarantees memory ordering and buffer consistency via strict execution sequencing. Decoupled command pairs (PIM_RdBuf/PIM_WrBuf and PIM_LdBuf/PIM_StBuf), derived from single CPU-mediated data transfers, must execute in program order, enforced by the memory scheduling strategy in Section 6. While other concurrent commands (PIM execution and CPU access) can be interleaved with the decoupled command pairs without ordering constraints since they never access the dedicated PIM buffer regions. These commands can arbitrarily interleave with the data transfer stream, even between decoupled pairs, without violating correctness.

6. Idleness-aware Memory Scheduling

This section describes our idleness-aware memory scheduling strategy designed to enhance PIM performance under the CPU-first scheduling principle. The strategy is implemented through two key mechanisms. First, IWE dynamically computes idle time windows for each DRAM bank and the memory bus by analyzing the CPU request queue. This enables more accurate scheduling decisions by the PIM scheduler and Command Arbiter. Second, utilizing the metadata provided by IWE, the PIM scheduler selects commands that optimize the utilization of these idle time windows in both the memory bus and the banks, while simultaneously maintaining low latency for CPU requests.

6.1. Idle Window Estimator (IWE)

6.1.1. Bank Idle Time Window Estimation. The occurrence of idle time windows in DRAM banks can be attributed to two primary factors. First, the CPU is generally unable to maintain consecutive memory requests in rapid succession due to the sporadic nature of memory access at the application level, resulting in idle time windows between accesses to the same bank. These inter-request idle time windows have already been leveraged by previous CPU-first scheduling algorithms [39]. Such schedulers initiate PIM commands when the CPU request queue for a bank is empty and promptly suspend PIM operations upon the emergence of any new CPU request.

The second type of idle time window occurs when, despite the availability of multiple banks with open rows prepared for access, serialization of their access commands over a shared memory bus leads to periods of inactivity. Specifically, this inactivity occurs between the activation of a row (ACT) and its designated data access, during which the bank remains idle with the row open, resulting in the internal bandwidth waste

highlighted in Fig. 3(c). This interval can be strategically optimized by delaying the ACT command until immediately before the data access. In the COSM architecture, the IWE module forecasts the earliest feasible service time for subsequent CPU access of each bank, based on the pending CPU requests. Using this information, the IWE module instructs the Command Arbiter to postpone premature row activations. IWE module uses the resulting idle time window in the bank for PIM operations. As illustrated in Fig. 3(e), the delayed row activation creates a window that can accommodate an additional PIM execution, improving the utilization of internal bandwidth.

IWE estimates the earliest time the next CPU request will be issued for each bank by simulating the FR-FCFS scheduling order of pending requests, thereby estimating each bank's future idle time window. Although the current prototype is designed for FR-FCFS scheduling, IWE module is intrinsically adaptable to different scheduling policies, via modifications to its estimation logic that align with alternative baseline scheduling policies (e.g., [58,59]). Because the request queue can dynamically change as new requests arrive or CPU memory accesses are completed, IWE must provide rapid predictions. Therefore, IWE leverages two key characteristics of FR-FCFS. First, row-hit requests are processed consecutively, since they bypass row activation and only require the memory bus (i.e., the external bandwidth), preempting requests from other banks. Secondly, requests within the same rank are grouped to prevent rank switch penalties (t_{RTS}), deferring inter-rank switching until there are no more ready requests in the current rank. These insights allow IWE to closely approximate the actual scheduling order with small overhead, forming the foundation of our algorithm that estimate the earliest access cycle of each bank (Algorithm 1).

Algorithm 1 Earliest Access Cycle Estimation in IWE

Require: $REQ[]$ (Earliest-arriving request of each bank)

```

1:  $ready\_cycles \leftarrow [get\_ready\_cycle(r) \text{ for } r \text{ in } REQ]$ 
2:  $t \leftarrow cur\_tick(), cr \leftarrow cur\_rank(), service\_time \leftarrow \{\}$ 
3: while  $REQ.size() > 0$  do
4:    $AnyReady \leftarrow Any([r.rank == cr \ \&\& \ r.ready \leq t \text{ for } r \text{ in } REQ])$ 
5:   if  $AnyReady$  then
6:      $r \leftarrow earliest\_ready([r \text{ for } r \text{ in } REQ \mid r.rank == cr])$ ,  $t \leftarrow t + t_{BL}$ 
7:   else
8:      $r \leftarrow earliest\_ready(REQ)$ 
9:      $t \leftarrow \max(r.ready, t)$ ,  $cr \leftarrow r.rank$ 
10:     $service\_time[r] \leftarrow t$ ,  $REQ.remove(r)$ 
11: for  $b$  in  $range(bank\_num)$  do
12:    $window\_bank[b] \leftarrow service\_time[r] \mid r.bank == b$ 
13:  $window\_bus \leftarrow \min(service\_time.values)$ 
14: return  $window\_bank, window\_bus$ 
```

In this algorithm, IWE estimate the earliest service time of each bank's earliest-arriving request to obtain the idle windows. In line 1, IWE calculate the ready cycles of each bank's earliest-arriving request ($REQ[]$) according to the bank state. Each bank can be in one of the three states: (1) Row-Closed,

which requires an ACT command, delaying the access by at least t_{RCD} from the current cycle; (2) *Opened-to-the-target-row*, which permits the access t_{RCD} after the row's opening cycle; (3) *Opened-to-a-different-row*, which requires both PRE and ACT commands, deferring the access past $t_{RP} + t_{RCD}$ from the current cycle. In line 2, the algorithm initializes t to current tick and cr to the rank of the previously issued command. Then, in the loop, it selects the earliest ready request in rank cr at tick t for issuing, and advances t by t_{BL} (line 6). If none is found, a rank switch occurs: the earliest request from the whole list is chosen, updating cr to its rank (lines 8-9). After processing all requests, IWE determines each bank's idle time window according to the earliest service cycle of each bank (line 12).

Idleness-Aware Command Arbitration. The Command Arbiter utilizes idle time window estimates from IWE to guide two primary choices for command scheduling decisions. First, when the CPU memory scheduler issues a ACT command, the Command Arbiter evaluates if the estimated idle time window for a bank is long enough for a PIM operation to take place. This window needs to encompass the time required for row switching ($t_{RP} + t_{RCD}$) and must also permit the PIM unit to process at least one column. If these conditions are met, the ACT command is deferred to allow for PIM execution. Second, if the CPU request queue for a bank becomes non-empty while a PIM unit is executing, the Command Arbiter refrains from issuing a PIM_Pause immediately. Rather, it delays the pause until the last possible cycle that would ensure the next CPU memory access is not delayed, maintaining the CPU-first principle.

6.1.2. Memory Bus Idle Window Estimation. Alongside estimating idle time windows for each bank, IWE estimates idle time windows on the memory bus. This allows the PIM scheduler to issue external transfer PIM commands (PIM_RdBuf and PIM_WrBuf), without hindering CPU accesses. The estimation utilizes the earliest service times of all outstanding CPU requests (Line 13 of Algorithm 1). The memory bus will remain idle from the present cycle until cycle *window_bus*. As a result, any external transfer PIM command that can be completed before this cycle can be issued without delaying CPU access.

6.2. PIM Scheduler

The PIM scheduler dynamically chooses the best PIM command from PRWQ and PEQ to utilize idle time windows. In conventional PIM execution, depicted in Fig. 7(a), the process includes three sequential stages: input data transfer, PIM execution, and result collection, all managed by the CPU software sequentially. This sequential processing prevents simultaneous usage of both internal bandwidth (①) and external bandwidth (②).

To avoid this sequential execution and improve both internal and external bandwidth usage, we propose an *overlapped scheduling strategy* within the PIM scheduler. Unlike the traditional method where each stage must finish entirely before the next can begin, our strategy divides the PIM workload into loosely-coupled tiles (e.g., submatrices in matrix multiplication). This allows CPU-mediated transfers of one tile to coincide with the PIM execution of another, since there are no data dependencies between the tiles (the CPU handles the

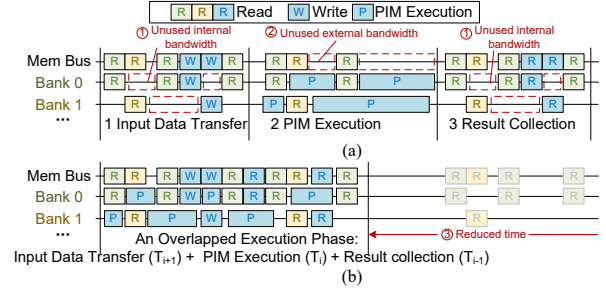


Fig. 7: (a) Conventional software-controlled three-stage sequential scheduling. (b) Overlapped scheduling. The row open operations are omitted for visual simplicity.

reduction). As shown in Fig. 7(b), the PIM scheduler issues compatible commands from different tiles *simultaneously* (e.g., collecting results for tile T_i while executing tile T_{i+1}), creating an *overlapped execution phase*. This approach maximizes the use of both internal and external bandwidth, thereby enhancing the end-to-end performance of PIM workloads (③). To guarantee that data dependencies are satisfied, a PIM_Barrier command is introduced between the overlapped execution phases, ensuring that all operations of one phase are completed before the next phase starts.

During each overlapped execution phase, the PIM scheduler chooses a PIM command from the PRWQ or the PEQ, based on a priority policy. The top priority is to issue a PIM_RdBuf or PIM_WrBuf command from PRWQ, as these commands solely occupy the external memory bus, which is often the bottleneck. Hence, it is crucial to efficiently utilize such external bus idle times. If no bus command is ready, the scheduler gives precedence to PIM_LdBuf and PIM_StBuf to ensure that the data is promptly loaded from or stored into the PIM buffer, preventing the blockage of subsequent bus commands. Note that our scheduler preserves arrival order for PRWQ commands targeting the same bank, correctly handling any ordering requirements between different requests. Specifically, when the reserved buffer segment is full during writing, the next issued command is necessarily PIM_StBuf that clears the buffer. Only when no PRWQ commands can be sent will the scheduler opt for a PIM execution command from the PEQ that matches the current idle period of the bank.

7. Effectiveness on the Software Stack

The COSM framework maintains programmer transparency through compiler-assisted command translation while maintaining backward compatibility with existing PIM programs that target command-driven PIM architecture [50,51]. Conventional PIM kernels (e.g., MAC and softmax) are directly translated into preemptable execution commands (PIM_Exec(L/S)) without modifying user-level code or computational semantics. CPU-mediated data transfers are automatically decomposed into adjacent command pairs: write sequences use PIM_WrBuf & PIM_StBuf while read sequences employ PIM_LdBuf & PIM_RdBuf (Section 5.2). To enable overlapped scheduling, the compiler automatically inserts PIM_Barrier commands at tile boundaries, ensuring the correctness of concurrent execution of data transfers and computations across independent

tiles.

As our solution requires no changes to program code or OS, no additional data-transfer overhead is incurred. This preserves the inherent advantage of LLM inference workloads, where weight matrices are pre-organized in banks during model loading and remain static throughout inference, following established practices in PIM-accelerated systems [19,23]. The primary change COSM introduces occurs at the memory controller driver level, which requires support for the new command semantics and scheduling interfaces described in Section 5.

8. Evaluation

8.1. Experimental Setup

Simulation. The performance of COSM and baselines is evaluated with the Ramulator2 simulator [60,61] (based on Ramulator [62]). The PIM commands and related timing constraints to the modeled LPDDR5 module. We extend the memory controller model with the additional modules described in Section 4. We use DRAMPower [63] to evaluate the power consumption of DRAM. Table 2 summarizes the modeled system configuration. The host CPU configuration is based on a mobile phone with a Qualcomm Snapdragon 888 [64] and 32GB of DRAM memory. The PIM unit to DRAM bank bandwidth and energy consumption are modeled on a taped-out PIM chip [36]. We use the O3CPU front-end model of Ramulator2 [60,61] for CPU simulation, with the CPU workload input to the front-end as a memory trace. For CPU workloads from SPEC CPU2017 and PolyBench-ACC, we use the zsim simulator [65] to generate the memory traces. A memory trace contains the memory accesses and the corresponding latency between the accesses. For mobile phone applications, we use the Xiaomi Mi 11 Pro [66] smartphone to run the workload and collect memory traces with instrumentation tools, Frida [67]. The Xiaomi Mi 11 Pro is equipped with a Qualcomm Snapdragon 888 SoC [64] (1 x 2.84 GHz Cortex-X1 + 3 x 2.42 GHz Cortex-A78 + 4 x 1.8 GHz Cortex-A55), 12GB LPDDR5 RAM [43], and 256GB storage, running Android 15 [68]. The DRAM configuration follows LPDDR5-6400 standards [43]. The CPU memory traces are repeatedly replayed to generate background DRAM traffic. To evaluate COSM’s area overhead, we synthesize the hardware modules in the memory controller using the Synopsys Design Compiler with TSMC 90nm technology library at the frequency of 2.4GHz. Since modern smartphone SoCs are typically fabricated in processes below 5nm, we provide a conservative area estimate for a 5nm implementation by scaling logic and memory components separately, based on publicly available technology scaling data.

CPU Benchmarks. We evaluate COSM using six mobile applications, three PolyBench benchmarks [69], and three SPEC CPU2017 [70] specifically selected for their diverse memory access patterns. We use the following application workloads:

- *Tencent Meeting (TM)*: Video conference app; trace collected during whiteboard sharing.
- *Browser*: Default mobile web browser; trace collected during page loading.
- *X*: Social network; trace collected when clicking an article.

Table 2: System Configuration

Host CPU	
Processor	8 × CPU cores @2.1GHz on average Out of Order, 160 entry RoB, 4 IPC
L3	4MB, Assoc: 8, 64B Cache Line Size
CPU Scheduler	FR-FCFS [53,54]
PIM Queue	PEQ size: 2, PRWQ size: 2 (per bank)
DRAM	
DRAM	LPDDR5-6400, 8GB/Rank
Organization	Bank / Bank Group / Row / Column 4 / 4 / 16384 / 2048B
Timing Param. (ns)	tBL=2.5 (16), tRCD=tRP=4.7, tCL=6.3, tRAS=10.7, tRRD=1.3, tRFC=87.5, tWR=8.8, tWTR=3.1, tRTP=1.3, tCS=0.6, tREFI=967.5
PIM Units	
PIM Core	1GHz, 6.4 GB/s bandwidth 6.4TFLOPS [36] 1kB Buffer for CPU-mediated transfer 16-bit PIM-bank wire width
Num	16 per Rank, at Bank level
System Configuration	
CPU System	2 Channels × 2 Ranks with PIM units

- *Note*: Note-taking app; trace collected during typing.
- *YouTube*: Video streaming platform; trace collected during video watching.
- *Music*: System music streaming platform; trace collected during song playback.

PolyBench benchmarks show dense, contiguous memory access, leading to high memory bandwidth utilization and high row hit rates, while SPEC CPU2017 targets broader testing. Table 3 present the row hit rate of the evaluated benchmarks.

Table 3: Row Hit Rate of Benchmarks.

Bench.	Rate	Bench.	Rate	Bench.	Rate
TM	0.039	YouTube	0.187	Floyd	0.908
Browser	0.434	Music	0.010	519.LBM	0.843
X	0.056	Ludcmp	0.870	511.Povray	0.596
Note	0.011	Covariance	0.001	520.Omnetpp	0.958

PIM Benchmarks. We tested inference on mobile devices using three open-source LLMs, each with around one billion parameters: BLOOM-1B1 [71], DeepSeek-R1-1.5B [52], and Qwen2-0.5B [72]. BLOOM-1B1 is a multilingual model, DeepSeek-R1-1.5B specializes in programming, and Qwen2-0.5B is a compact bilingual model. Despite differences, all are suitable for mobile deployment. Benchmarks use 16-bit quantized inputs and 8-bit quantized weights.

Baselines. We compare COSM that use three baselines with different PIM control interfaces and scheduling strategies: *All-Bank Command* [42], *Chopim* [39], and *AsyncDIMM-Bank*, which is the bank-level PIM version of the original *AsyncDIMM* [38]. In *All-Bank Command*, PIM operations are issued using all-bank commands that simultaneously invoke all PIM units. Since CPU memory access is blocked after issuing the all-bank PIM commands, we introduce a time-sliced round-robin strategy: assume that 95% of the time is spent on CPU memory access, while the remaining 5% is allocated to PIM computation: we consider CPU performance degradation

to be small enough as long as it is smaller than a 5% threshold. Crucially, to ensure a fair and conservative comparison, we assume idealized zero-overhead switching between PIM and CPU phases in this baseline. This isolates the impact of the memory interface architecture on scheduling potential. In contrast, *Chopim* and *AsyncDIMM-Bank* adopt single-bank commands with the PIM execution command length equal to t_{BL} . *Chopim* adopts a CPU-first scheduling strategy, which prioritizes CPU memory accesses over PIM computation by blocking the PIM command queue whenever the CPU memory queue of a bank is not empty. *AsyncDIMM-Bank* adopts a relatively fair strategy to maximize the row hit rate by switching between the PIM command queue and CPU memory queue upon detecting a PRE command for either CPU or PIM unit. Moreover, it uses a relay memory controller within the rank to reduce command bus pressure. None of the baselines mentions their scheduling strategy of CPU-mediated data transfer. We assume that these memory accesses are scheduled together using the FR-FCFS policy. COSM adopts both the low-interference PIM control interface and the idleness-aware scheduling strategy. We set the PIM execution command length to 128 cycles, a value that balances the command bus contention (if too short) against excessive PIM_Pause commands and their bus bandwidth consumption (if too long), as observed in our experiments.

8.2. Overall Performance

Fig. 8 shows the performance of COSM’s CPU and PIM benchmarks during simultaneous execution compared to the baselines. CPU performance is normalized to the performance obtained under standalone execution, which indicates the ideal case. The key-value cache is configured to a length of 2k. We also include a baseline called *Chopim(128)* by increasing the PIM execution command length of *Chopim* to 128 cycles. For the *All-bank* baseline, the PIM units are restricted to using only 5% of the time windows, missing the opportunity to take advantage of the idle periods between CPU memory accesses. The CPU-first approach of *Chopim* results in a small 3.0% slowdown for the CPU but only provides a relatively small 1.9 $\times$ increase in PIM throughput over *All-Bank*, due to command-bandwidth contention. *AsyncDIMM-Bank* provides a 4.2 $\times$ PIM throughput compared to *All-bank*, yet it significantly affects

CPU performance, reducing it by an average of 89.9%. Although increasing the PIM execution command length to 128 cycles can increase the PIM throughput of *Chopim* by 3.44 $\times$, the CPU performance reduction increased to 13.5%, indicating the trade-off between PIM performance and CPU performance for fixed-length commands. In contrast, the scheduling strategy of COSM maintains CPU performance with only a 2.0% degradation. Additionally, the low-interference PIM control interface and idleness-aware scheduling enable COSM to more effectively use the remaining internal bandwidth for PIM tasks, leading to a 6.0 $\times$ improvement over the *All-Bank* baseline and 2.8 $\times$ over *Chopim*.

8.3. Effect of Preemptable PIM Execution

Fig. 9 illustrates the performance of CPU and PIM benchmarks during concurrent execution, comparing traditional fixed-length commands across different lengths (from 16 to 256), against our proposed preemptable PIM execution command. Both PIM and CPU performance is also normalized to performance of standalone execution. The maximum command length of 256 remains below the latency threshold for processing a complete row in our setup. The PIM workload corresponds to a KQV generation layer from the three models that is dominated by PIM execution. For fixed-length commands, there is a clear trade-off between CPU and PIM performance: longer commands enhance PIM workload throughput but negatively impact CPU performance. Notably, when the PIM execution command length surpasses 64 cycles, PIM performance reaches saturation in most cases due to decreased command bus pressure. This phenomenon is aligned to Fig. 2(b). However, CPU performance significantly declines by over 5%, particularly in mobile applications with random access patterns. In contrast, our preemptable interface can achieve maximum PIM performance without with only 3.2% degradation in CPU performance. Compared to a fixed command length of 32 cycles, which maintains CPU interference below 5% for all workloads, our preemptable PIM execution design results in a 2.02 $\times$ improvement in PIM performance.

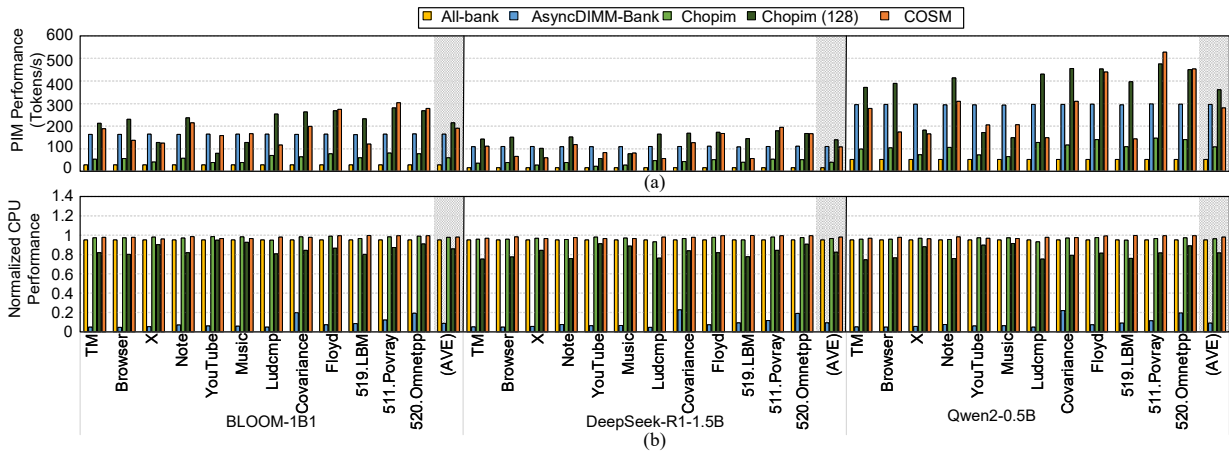


Fig. 8: Overall PIM & CPU performance of COSM and baselines for concurrent CPU and PIM execution.

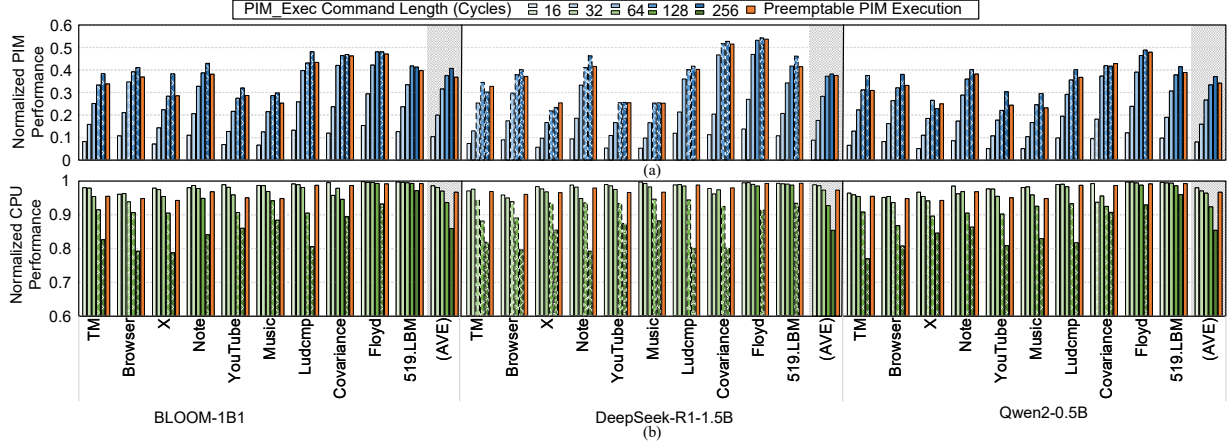


Fig. 9: Normalized CPU and PIM workload performance under fixed-length and preemptible PIM execution command. Cases where CPU performance degrades by more than 5% are marked with dashed lines.

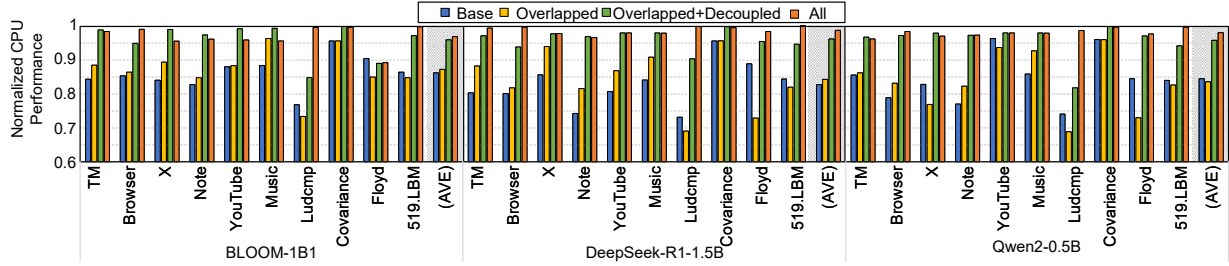


Fig. 10: Impact of CPU-mediated data transfers on CPU performance under different scheduling strategies. We test on the attention layers of the benchmarks.

8.4. Sensitivity Analysis on CPU Performance during CPU-mediated Data Transfer

To assess how our techniques affect CPU-mediated transfers, we evaluate CPU performance normalized to the standalone execution, under four different configurations using attention layers from the three models that are characterized by substantial CPU-mediated transfer, as depicted in Fig. 10. To assess how our techniques affect CPU-mediated transfers, we evaluate the normalized CPU performance under four configurations using attention layers from three models characterized by substantial CPU-mediated transfers, as depicted in Fig. 10. In the *Base* configuration, CPU-mediated transfers occur through standard read/write requests, following the traditional three-stage PIM execution. *Overlapped* execution facilitates the simultaneous scheduling of CPU-mediated transfers and PIM execution commands by utilizing our scheduler design described in Section 6.2. This approach increases CPU performance by an average of 3.7% across mobile workloads compared to *base*. However, for workloads with high row-hit rates (such as algebraic kernels in PolyBench-ACC), this aggressive overlap can inadvertently cause additional row switches, leading to greater CPU interference. The *Overlapped+Decoupled* configuration further employs bandwidth-decoupled CPU-mediated PIM data transfer commands (Section 5.2), which reduces contention with CPU accesses and increase CPU performan by 11.5% compared to *Base*. Finally, the *All* configuration utilizes IWE to proactively detect memory bus idle time windows that are sufficiently large to accommodate entire burst transfers,

thereby maintaining CPU slowdown consistently below 5% across all evaluated workloads.

8.5. Effect of Idleness-aware Scheduling on PIM Execution

In Fig. 11(a), we illustrate the performance of PIM workloads when managed by CPU-first scheduling compared to idleness-aware scheduling (Section 6) when running alongside with CPU workloads, with all results normalized to their standalone execution. The PIM workload is derived solely from the execution segment of an attention layer, excluding CPU-mediated data transfer, from the DeepSeek model. Under CPU-first scheduling, the issuance of PIM execution commands halts if a bank queue is not empty. In the idleness-aware configuration, idle time windows are predicted using IWE to enhance performance. This configuration provides an average performance boost of $1.21\times$ for PIM workloads across mobile applications. Fig. 11(b) displays the bandwidth usage under both scheduling strategies. The term “available bandwidth” refers to idle time windows that are long enough for a PIM execution command but are not used due to scheduling limitations, whereas “unavailable bandwidth” pertains to short, fragmented idle periods unsuitable for PIM execution. Compared to CPU-first scheduling, Idleness-Aware scheduling exploits an additional 37.0% of the available bandwidth, leaving less than 1% unused.

8.6. Energy Consumption

Fig. 12 depicts energy consumption per token of COSM and baselines. We only calculate the energy of PIM computation,

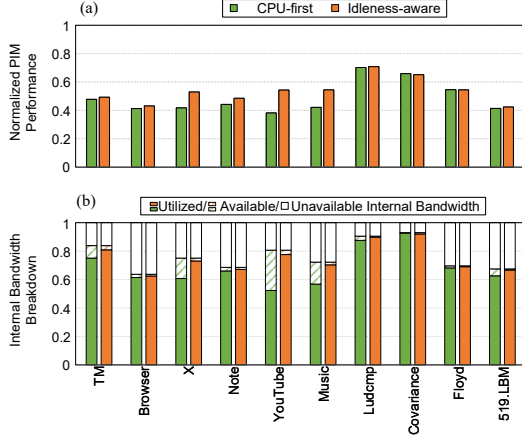


Fig. 11: (a) PIM performance and (b) Internal bandwidth usage during concurrent execution with CPU workload under CPU-first scheduling and idleness-aware scheduling.

and exclude the energy of CPU access during concurrent execution. Although COSM’s fine-grained CPU-PIM task interleaving increases row switching overheads, it achieves a reduction in total energy by eliminating long active-idle periods. COSM reduces energy by $1.34\times/1.61\times$ compared to the AsyncDIMM-Bank/Chopim baseline.

8.7. Sensitivity Analysis

PIM Execution Command Length ($nPTL$). Fig. 13(a) analyzes the sensitivity of COSM to the $nPTL$ timing constrain. Generally, smaller command length ($nPTL < 32$) saturates the command bus, exacerbating contention between CPU and PIM tasks. However, moderately reducing $nPTL$ can yield benefits by decreasing PIM_Pause injections that potentially stall CPU accesses. For instance, in workload X, reducing $nPTL$ from 128 to 64 improves both CPU and PIM performance.

Rank Count per Channel. Fig. 13(b) analyzes the PIM performance of COSM to the rank count in each channel. Higher rank counts require the memory controller to issue more commands to trigger the execution of all PIM units, making the command bus easily saturated. Therefore, scaling $nPTL$ from 32 to 128 yields a higher speedup for 4 ranks ($1.27\times$) than for 1 rank ($1.22\times$).

KV Cache Size. Fig. 13(c) depicts the CPU performance degradation of COSM to the KV cache size of the DeepSeek attention layer, adopting the same series labels as Fig. 10. As

the KV cache grows, CPU-mediated data transfer increasingly dominates the total computation time. This mitigates the performance drop in the *Base* case, which is highly sensitive to data transfer overhead. In contrast, COSM effectively manages both data transfer and PIM execution, maintaining CPU performance degradation below 5%.

tRP . Fig. 13(d) shows performance under varying tRP (row precharge time) using a DeepSeek attention layer, excluding CPU-mediated data transfer. Under the current timing configuration (listed in Table 2), the CPU-PIM interference remains minimal. Since CPU and PIM requests typically map to different rows, a larger tRP heightens the row-switch penalty between CPU access and PIM execution command. This amplifies interference and degrade both CPU and PIM performance.

8.8. Area Overhead.

Implemented in a Snapdragon 8-class LPDDR5 memory controller (estimated at 0.93 mm^2 , measured from publicly available die photographs), the COSM hardware modules occupy 0.069 mm^2 , resulting in a modest 7.4% area overhead. This includes 0.014 mm^2 for the PIM scheduler, 0.0085 mm^2 for the IWE, and 0.0054 mm^2 for the Command Arbiter.

9. Discussion

Applicability to Broader Architectures. Although evaluated on mobile platforms, COSM’s idleness-aware scheduling is applicable to any shared-memory CPU-PIM system. As PIM extends to desktops, servers, data centers, and supercomputers for diverse workloads [17,23,31,32,73], they inherently encounter the same bank and bus contention challenges appear when CPU and PIM units contend with each other. COSM addresses this through a universal memory abstraction: PIM workloads are characterized as bandwidth-sensitive, whereas CPU workloads are latency-sensitive

Extension to Heterogeneous Accelerators. Although COSM targets CPU-PIM coordination, supporting heterogeneous agents (e.g., GPU/NPU) is vital. With memory-bound tasks offloaded to PIM, these accelerators handle compute-intensive tasks and remain latency-sensitive. COSM’s principle applies directly: located in the memory controller, it treats accelerator accesses as standard host requests, effectively isolating them from PIM-induced delays. Given the unique access patterns and scheduling policies of accelerators, extending COSM to explicitly capture these characteristics is a key research direction for future work.

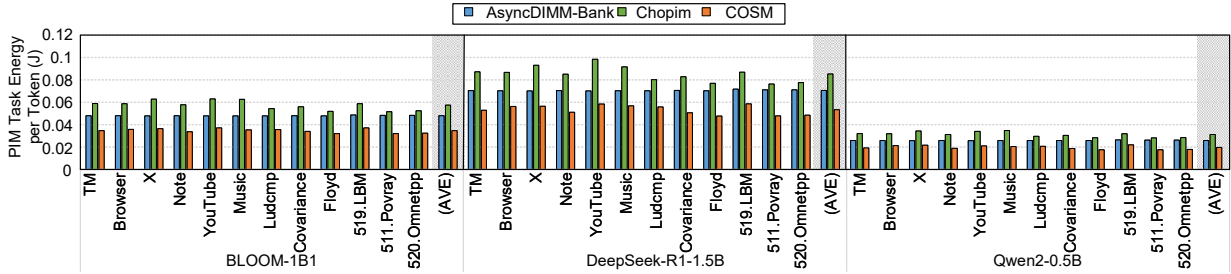


Fig. 12: PIM workload energy consumption per token (including PIM unit computation, PIM bank access, and CPU-mediated data transfer) of COSM and baselines during concurrent CPU and PIM execution.

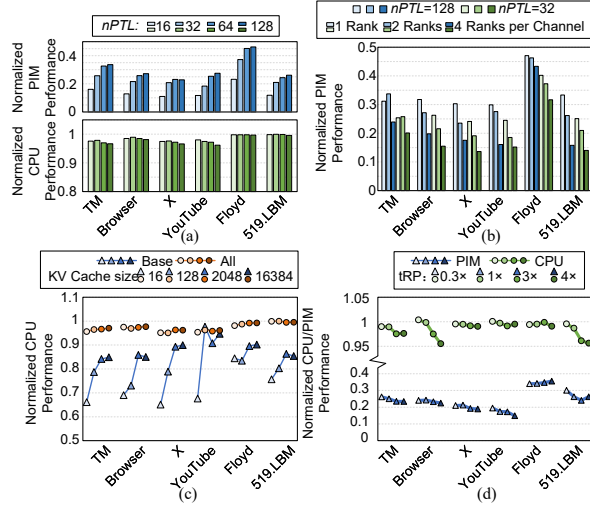


Fig. 13: Sensitivity analysis on (a) $nPTL$ (b) rank count per channel (c) KV cache size (d) scaled tRP ($n\times$, relative to Table 2 configuration).

Compatibility with SIMD PIM Architectures While COSM currently targets single-bank PIM, its core principles readily extend SIMD PIMs (e.g., bank-group- or subarray-level) [74–78]. However, the inherent parallelism of SIMD execution poses challenges to idle time window prediction, as it requires the simultaneous availability of multiple banks. Consequently, future adaptations must balance PIM parallelism with CPU latency guarantees. Our scheduler enables this extensibility, but a dedicated exploration is left for future work.

Thermal Considerations. While COSM maximizes PIM throughput by exploiting CPU idle time windows, it can support thermal management. First, Command Arbiter can throttle PIM if the temperature exceeds thresholds. Second, IWE can reduce idle time window utilization, explicitly trading PIM throughput for thermal safety. Future work could explore adaptive strategies driven by real-time thermal feedback.

10. Related Work

PIM Architecture. PIM [31] integrates computational units inside memory devices to improve the performance of memory-intensive workloads. This design philosophy applies universally across the entire storage hierarchy, including SRAM cache [79–85], DDR [15,27,29,86,87], GDDR [16], HBM [42], 3D stacked memory [88–90], and SSD [91–97]. Driven by this media-wide adaptability, researchers have proposed numerous application-specific PIM microarchitectures tailored for diverse workloads [28,42,76,87,94,98–102].

In DRAM-based PIM systems, the absence of direct inter-PIM-unit interconnects forces data transfers to be mediated by CPU, incurring prohibitive communication overheads. To bridge this gap, some works propose to introduce additional connections or custom access modes [103–107]. However, these methods introduce additional hardware complexity and require non-trivial adaptations at the DRAM interface. Enabling PIM compatibility with commodity CPU ecosystems has also emerged as a major research focus. To achieve this, subsequent works optimize system-level compatibility through

software or protocol layers, such as deploying dynamic address mapping for shared memory spaces [18,37], partitioning dedicated memory regions [15,42], or implementing specialized cache coherence frameworks [86,108,109]. These system-level protocols inevitably strain the memory control interface due to frequent host-PIM interactions. To address this, frameworks like AsyncDIMM [38] and ComPASS [41] further alleviate host command issuance pressure and scheduling overheads. Importantly, COSM operates orthogonally to these system and control interface optimizations, focusing on low-interference scheduling to further maximize overall system efficiency.

Memory Scheduling. Conventional memory scheduling primarily focuses on mitigating row switching latencies [44,110,111] and read-write delays [112,113]. To scale within multi-core systems, recent works exploit individual thread variations to achieve fair and high-performance scheduling [56–59,114–122], while also enforcing deadline-aware constraints [123–125]. As architectures shift toward heterogeneous systems (e.g., CPU-GPU), scheduling evolves to manage highly asymmetric core behaviors and requirements [126–130]. To address unpredictable dynamic patterns, another research path deploys learning-based methods to adaptively optimize scheduling [131–133]. For PIM systems, the core difference is that CPU and PIM units do *not* share a unified interface, operating via separate command sets that cause severe cross-domain interference and necessitate dedicated co-scheduling. We have analyzed the two extreme policies in Section 3.4, CPU-first [39] and row-hit-aware [38,40], which biasedly favor CPU and PIM performance, respectively. ComPASS [41] introduces batch scheduling to balance performance, yet it enforces coarse-grained time-slicing between CPU and PIM, failing to achieve concurrent execution. COSM simultaneously satisfies the low-latency requirements of CPU accesses while maximizing PIM throughput, minimizing interference during concurrent execution.

11. Conclusion

This paper introduces COSM, a cooperative memory access scheduling framework that accelerates LLM inference on mobile devices with marginal degradation to CPU performance. To achieve this, COSM tackles CPU-PIM resource contention from both the control interface and memory scheduling perspectives, dynamically leveraging host memory idle windows for PIM execution. Consequently, COSM achieves a $2.8\times$ increase in LLM inference throughput with less than 2.0% CPU performance degradation during concurrent execution.

Acknowledgments

We sincerely thank the anonymous reviewers for their comments and suggestions to improve the paper. This work was partially supported by the National Key Research and Development Program of China (2024YFE0204300), National Natural Science Foundation of China (Grant No.62402311), Natural Science Foundation of Shanghai (Grant No.24ZR1433700), and Key Research and Development Program of Shanghai (25LN3201200).

A. Artifact Appendix

A.1. Abstract

This artifact provides the complete experimental framework for COSM, a cooperative scheduling framework designed for concurrent PIM/CPU execution on mobile devices. The artifact is built on the Ramulator2 simulator, which has been extended with the proposed low-interference PIM control interface and idleness-aware scheduling methodology.

The software components include the modified C++ source code of the Ramulator2-based memory controller, specialized PIM instruction support, and the scheduling logic for PIM Execution Queues (PEQ) and PIM Read/Write Queues (PRWQ). To represent realistic mobile scenarios, we provide two types of memory traces: 1) LLM workload traces (e.g., DeepSeek-R1-1.5B) generated via Python scripts to simulate PIM tasks, and 2) Mobile application traces (e.g., Browser, YouTube) collected from physical mobile devices to represent CPU tasks.

This artifact is designed to reproduce the key results presented in the paper, specifically the performance improvements and CPU interference analysis shown in Figures 8 and 10. Users can expect to observe that COSM significantly enhances PIM throughput (up to 2.8 $\times$) while maintaining minimal CPU performance loss (less than 5.0%). The artifact requires a GCC 12+ compiler for building the simulator environment.

A.2. Artifact check-list (meta-information)

- **Algorithm:** Idleness-aware PIM scheduling, low-interference PIM control interface, and PIM/CPU concurrent memory access management.
- **Program:** Modified Ramulator2 simulator (C++), trace generation scripts (Python).
- **Compilation:** GCC 12 or higher (supporting C++20), CMake (3.30+).
- **Data set:** LLM execution traces (DeepSeek-R1 series) and real-world mobile application traces (e.g., Browser, YouTube, Camera) collected from mobile devices.
- **Run-time environment:** Linux (Ubuntu 20.04/22.04 recommended), Python 3.12+.
- **Hardware:** Any x86_64 CPU for simulation; approximately 64GB RAM recommended for large trace handling.
- **Execution:** Automated simulation via Python task scripts; manual data integration into Excel templates.
- **Metrics:** PIM throughput (Tokens/sec), CPU performance slowdown.
- **Output:** Simulation log files, CSV tables, and generated plots (matching Figures 8, 10).
- **Experiments:** Baseline vs. COSM comparison under different mobile scenarios; sensitivity analysis of PEQ/PRWQ sizes and scheduling thresholds.
- **How much disk space required (approximately)?:** 5-10 GB (mostly for memory traces).
- **How much time is needed to prepare workflow (approximately)?:** 30-60 minutes (for environment setup and simulator compilation).
- **How much time is needed to complete experiments (approximately)?:** 1-2 hours.
- **Publicly available?:** Yes (via Zenodo).
- **Workflow automation framework used?:** Bash scripts.
- **Archived (provide DOI)?:** <https://doi.org/10.5281/zenodo.19660293>

A.3. Description

A.3.1. How to access. The artifact is available via Zenodo. The repository includes the modified Ramulator2 source code, Python execution scripts, sample traces, and the Excel visualization template.

A.3.2. Hardware dependencies. The artifacts are based on software simulation. To reproduce the results within a reasonable timeframe (as specified in the check-list), a multi-core x86_64 CPU (8+ cores) and at least 64GB of RAM are recommended.

A.3.3. Software dependencies. The simulator requires GCC 12 or higher to support C++20 standards. The experiment orchestration and data collection rely on Python 3.12+. For data visualization, we provide an Excel template that processes the generated CSV files.

A.3.4. Data sets. We include a comprehensive suite of memory traces:

- **CPU Workloads:** Real-world mobile application traces (e.g., YouTube, Browser) captured from a Xiaomi 11 Pro [66].
- **PIM Workloads:** LLM inference traces (e.g., DeepSeek-R1-1.5B) generated to simulate PIM-side memory access patterns.

A.3.5. Models. Our model extends Ramulator2 with the following COSM-specific features:

- **Memory Controller:** Implements the Idleness-aware scheduling logic, specialized queues (PEQ/PRWQ), and PIM/CPU command arbitration. (src/dram_controller/impl/plugin/pim/pim_arbiter_cpu_first_o3_pred.cpp)
- **PIM Control Interface:** Adds support for low-interference PIM control commands. (src/dram/impl/LPDDR5-PIM.cpp)

A.4. Installation

The installation of COSM follows the standard build procedure of Ramulator2.

1. **Environment Setup:** The simulator requires a C++20 compliant compiler (GCC 12+) and CMake 3.30+. Detailed dependency requirements can be found in the official Ramulator2 documentation: <https://github.com/cmu-safari/ramulator2>. Alternatively, you can use the automated script provided in the repository:

```
./build.sh
```

You can specify the parallelism level (default: 8) by running:

```
./build.sh [parallelism]
# e.g. ./build.sh 16
```

2. **Verification:** To ensure the build is successful, verify that the ramulator2 executable is generated in the project root or the build/ directory. You can run ./build/ramulator2 --help to check the basic functionality.

A.5. Experiment workflow

The evaluation is conducted in three main stages: simulation execution, data aggregation, and visualization.

1. Simulation Execution: We use a bash script `run_script.sh` to automate the simulation execution. The script automatically manages the Ramulator2 execution and reads the PIM/CPU traces.

```
cd simulations/
./run_script.sh figure8
./run_script.sh figure10
```

Alternatively, you can specify the parallelism level (default: 8) by running:

```
./run_script.sh figure8 [parallelism]
# e.g. ./run_script.sh figure8 16
./run_script.sh figure10 [parallelism]
```

This will generate a CSV file under the `simulations/scripts_result_processor` directory.

2. Visualization: Due to the complexity of the micro-architectural data analysis, we provide three Excel templates (`plot_figure8|10_ae.xlsx`) under `simulations/scripts_result_processor` directory for final plotting.

- Open the CSV file generated by the simulation (`summary\_figure8|10.csv`) and the corresponding Excel template.
- Copy the content of the generated CSV file and paste it into the table **Summary**.
- The corresponding charts in table **Chart** will automatically update to reflect the results, reproducing the trends shown in the paper.

A.6. Evaluation and expected results

The evaluation focuses on comparing the performance of COSM against baseline scheduling policies under concurrent PIM/CPU execution scenarios. The reproduced results are expected to closely match the trends presented in the paper. COSM should consistently demonstrate higher PIM throughput (up to 2.8 $\times$) and lower CPU interference (within 5.0% CPU performance loss).

References

- [1] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, "Moshi: a speech-text foundation model for real-time dialogue," *arXiv e-prints*, p. arXiv:2410.00037, Sep. 2024.
- [2] Q. Chen, Y. Chen, Y. Chen, M. Chen, Y. Chen, C. Deng, Z. Du, R. Gao, C. Gao, Z. Gao, Y. Li, X. Lv, J. Liu, H. Luo, B. Ma, C. Ni, X. Shi, J. Tang, H. Wang, H. Wang, W. Wang, Y. Wang, Y. Xu, F. Yu, Z. Yan, Y. Yang, B. Yang, X. Yang, G. Yang, T. Zhao, Q. Zhang, S. Zhang, N. Zhao, P. Zhang, C. Zhang, and J. Zhou, "MinMo: A Multimodal Large Language Model for Seamless Voice Interaction," *arXiv e-prints*, p. arXiv:2501.06282, Jan. 2025.
- [3] Y. Shi, Y. Shu, S. Dong, G. Liu, J. Sesay, J. Li, and Z. Hu, "Voila: Voice-Language Foundation Models for Real-Time Autonomous Interaction and Voice Role-Play," *arXiv e-prints*, p. arXiv:2505.02707, May 2025.
- [4] M. Wang, J. Zhao, T.-T. Vu, F. Shiri, E. Shareghi, and G. Haffari, "Simultaneous Machine Translation with Large Language Models," *arXiv e-prints*, p. arXiv:2309.06706, Sep. 2025.
- [5] B. Fu, M. Liao, K. Fan, C. Li, L. Zhang, Y. Chen, and X. Shi, "LLMs Can Achieve High-quality Simultaneous Machine Translation as Efficiently as Offline," *arXiv e-prints*, p. arXiv:2504.09570, Apr. 2025.
- [6] C. Lugaresi, J. Tang, H. Nash, C. McClanahan, E. Uboweja, M. Hays, F. Zhang, C.-L. Chang, M. G. Yong, J. Lee, W.-T. Chang, W. Hua, M. Georg, and M. Grundmann, "MediaPipe: A Framework for Building Perception Pipelines," *arXiv e-prints*, p. arXiv:1906.08172, Jun. 2019.
- [7] M. Tanveer, Y. Zhou, S. Niklaus, A. Mahdavi Amiri, H. Zhang, K. K. Singh, and N. Zhao, "MotionBridge: Dynamic Video Inbetweening with Flexible Controls," *arXiv e-prints*, p. arXiv:2412.13190, Dec. 2024.
- [8] Apple Inc., "Apple Intelligence," <https://www.apple.com/apple-intelligence/>, 2024, accessed: 2025-11-05.
- [9] H. Chen, Y. Wang, K. Han, D. Li, L. Li, Z. Bi, J. Li, H. Wang, F. Mi, M. Zhu, B. Wang, K. Song, Y. Fu, X. He, Y. Luo, C. Zhu, Q. He, X. Wu, W. He, H. Hu, Y. Tang, D. Tao, X. Chen, and Y. Wang, "Pangu Embedded: An Efficient Dual-system LLM Reasoner with Metacognition," *arXiv e-prints*, p. arXiv:2505.22375, May 2025.
- [10] Y. Yao, T. Yu, A. Zhang, C. Wang, J. Cui, H. Zhu, T. Cai, H. Li, W. Zhao, Z. He *et al.*, "MiniCPM-V: A GPT-4V Level MLLM on Your Phone," *Nat Commun* 16, 5509 (2025), 2025.
- [11] Samsung Electronics, *Galaxy.ai - The #1 All-in-One AI Platform*, <https://galaxy.ai/>, 2024, accessed: 2025-11-05.
- [12] X. Lu, Y. Chen, C. Chen, H. Tan, B. Chen, Y. Xie, R. Hu, G. Tan, R. Wu, Y. Hu, Y. Zeng, L. Wu, L. Bian, Z. Wang, L. Liu, Y. Yang, H. Xiao, A. Zhou, Y. Wen, X. Chen, S. Ren, and H. Li, "BlueLM-V-3B: Algorithm and System Co-Design for Multimodal Large Language Models on Mobile Devices," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2025, pp. 4145–4155.
- [13] Qualcomm Technologies, Inc., "Unlocking On-Device Generative AI with an NPU and Heterogeneous Computing," Qualcomm, Tech. Rep., 2024, accessed: 2025-11-15.
- [14] vivo, "vivo Unveils New AI Strategy: BlueHeart Large Model Matrix and Major Upgrade to OriginOS 5," <https://www.vivo.com.cn/brand/news/detail?id=1271>, 2024, accessed: 2025-11-15.
- [15] F. Devaux, "The true Processing In Memory accelerator," in *2019 IEEE Hot Chips 31 Symposium (HCS)*, Aug 2019, pp. 1–24.
- [16] S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, N. Kim, Y. Kwon, K. Vladimir, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, J. Lee, D. Ko, Y. Jun, K. Cho, I. Kim, C. Song, C. Jeong, D. Kwon, J. Jang, I. Park, J. Chun, and J. Cho, "A 1nm 1.25V 8Gb, 16Gb/s/pin GDDR6-based Accelerator-in-Memory supporting 1TFLOPS MAC Operation and Various Activation Functions for Deep-Learning Applications," in *2022 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 65, Feb 2022, pp. 1–3.
- [17] Y. Zhao, M. Gao, H. Zhang, F. Liu, G. Chen, H. Xian, H. Guan, and L. Jiang, "PUSHtap: PIM-based In-Memory HTAP with Unified Data Storage Format," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '25, 2025, p. 179–194.
- [18] Y. Zhao, M. Gao, F. Liu, Y. Hu, Z. Wang, H. Lin, J. Li, H. Xian, H. Dong, T. Yang, N. Jing, X. Liang, and L. Jiang, "UM-PIM: DRAM-based PIM with Uniform & Shared Memory Space," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, June 2024, pp. 644–659.
- [19] Y. He, H. Mao, C. Giannoula, M. Sadrosadati, J. Gómez-Luna, H. Li, X. Li, Y. Wang, and O. Mutlu, "PAPI: Exploiting Dynamic Parallelism in Large Language Model Decoding with a Processing-In-Memory-Enabled Computing System," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, p. 766–782.
- [20] G. Heo, S. Lee, J. Cho, H. Choi, S. Lee, H. Ham, G. Kim, D. Mahajan, and J. Park, "NeuPIMs: NPU-PIM Heterogeneous Acceleration for Batched LLM Inferencing," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, ser. ASPLOS '24, 2024, p. 722–737.
- [21] J. Chen, Y. Qi, K. Sun, Z. Lin, T. Wang, C. Ma, and Y. Wang, "Unifying Two Operators with One PIM: Leveraging Hybrid Bonding for Efficient LLM Inference," in *Advanced Parallel Processing Technologies*, C. Li, X. Qian, D. Gizopoulos, and B. Grot, Eds., 2026, pp. 215–230.
- [22] W. Kim, Y. Lee, Y. Kim, J. Hwang, S. Oh, J. Jung, A. Huseynov, W. G. Park, C. H. Park, D. Mahajan, and J. Park, "Pimba: A Processing-in-Memory Acceleration for Post-Transformer Large Language Model Serving," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25, 2025, p. 292–307.
- [23] J. Park, J. Choi, K. Kyung, M. J. Kim, Y. Kwon, N. S. Kim, and J. H. Ahn, "AttAcc! Unleashing the Power of PIM for Batched Transformer-based Generative Model Inference," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '24, 2024, p. 103–119.
- [24] L. Yan, X. Lu, X. Chen, Y. Han, and X.-H. Sun, "Pyramid: Accelerating LLM Inference With Cross-Level Processing-in-Memory," *IEEE Computer Architecture Letters*, vol. 24, no. 1, pp. 121–124, Jan 2025.
- [25] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, "A scalable processing-in-memory accelerator for parallel graph processing," in *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, June 2015, pp. 105–117.
- [26] J. Ahn, S. Yoo, O. Mutlu, and K. Choi, "PIM-enabled instructions: A low-overhead, locality-aware processing-in-memory architecture," in *2015 ACM/IEEE 42nd Annual International Symposium on Computer Architecture (ISCA)*, June 2015, pp. 336–348.
- [27] A. Boroumand, S. Ghose, B. Akin, R. Narayanaswami, G. F. Oliveira, X. Ma, E. Shiu, and O. Mutlu, "Google Neural Network Models for Edge Devices: Analyzing and Mitigating Machine Learning Inference Bottlenecks," in *2021 30th International Conference on Parallel Architectures and Compilation Techniques (PACT)*, Sep. 2021, pp. 159–172.
- [28] M. Besta, R. Kanakagiri, G. Kwasniewski, R. Ausavarungnirun, J. Beránek, K. Kanellopoulos, K. Janda, Z. Vónarburg-Shmaria, L. Gianinazzi, I. Stefan, J. G. Luna, J. Goliński, M. Copik, L. Kapp-Schwörer, S. Di Girolamo, N. Blach, M. Konieczny, O. Mutlu, and T. Hoefler, "SISA: Set-Centric Instruction Set Architecture for Graph Mining on Processing-in-Memory Systems," in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '21, 2021, p. 282–297.
- [29] A. Boroumand, S. Ghose, Y. Kim, R. Ausavarungnirun, E. Shiu, R. Thakur, D. Kim, A. Kuesela, A. Knies, P. Ranganathan, and O. Mutlu, "Google Workloads for Consumer Devices: Mitigating Data Movement Bottlenecks," in *Proceedings of the Twenty-Third International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS '18, 2018, p. 316–331.
- [30] Y. Gu, A. Khadem, S. Umesh, N. Liang, X. Servot, O. Mutlu, R. Iyer, and R. Das,

- "PIM Is All You Need: A CXL-Enabled GPU-Free System for Large Language Model Inference," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, ser. ASPLOS '25, 2025, p. 862–881.
- [31] O. Mutlu, S. Ghose, J. Gómez-Luna, and R. Ausavarungrun, *A Modern Primer on Processing in Memory*, 2023, pp. 171–243.
 - [32] O. Mutlu, "Processing data where it makes sense in modern computing systems: Enabling in-memory computation," in *2018 7th Mediterranean Conference on Embedded Computing (MECO)*, June 2018, pp. 8–9.
 - [33] J. Gómez-Luna, I. E. Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking a New Paradigm: Experimental Analysis and Characterization of a Real Processing-In-Memory System," *IEEE Access*, vol. 10, pp. 52 565–52 608, 2022.
 - [34] J. Gómez-Luna, Y. Guo, S. Brocard, J. Legriel, R. Cimdado, G. F. Oliveira, G. Singh, and O. Mutlu, "Evaluating Machine Learning Workloads on Memory-Centric Computing Systems," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, April 2023, pp. 35–49.
 - [35] J. Gómez-Luna, I. El Hajj, I. Fernandez, C. Giannoula, G. F. Oliveira, and O. Mutlu, "Benchmarking Memory-Centric Computing Systems: Analysis of Real Processing-In-Memory Hardware," in *2021 12th International Green and Sustainable Computing Conference (IGSC)*, Oct 2021, pp. 1–7.
 - [36] J. H. Kim, Y. Ro, J. So, S. Lee, S.-h. Kang, Y. Cho, H. Kim, B. Kim, K. Kim, S. Park, J.-S. Kim, S. Cha, W.-J. Lee, J. Jung, J.-G. Lee, J. Lee, J. Song, S. Lee, J. Cho, J. Yu, and K. Sohn, "Samsung PIM/PNM for Transformer Based AI: Energy Efficiency on PIM/PNM Cluster," in *2023 IEEE Hot Chips 35 Symposium (HCS)*, Aug 2023, pp. 1–31.
 - [37] D. Lee, B. Hyun, T. Kim, and M. Rhu, "PIM-MMU: A Memory Management Unit for Accelerating Data Transfers in Commercial PIM Systems," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Nov 2024, pp. 627–642.
 - [38] L. Chen, D. Lyu, J. Jiang, Q. Wang, Z. Mao, and N. Jing, "AsynCDIMM: Achieving Asynchronous Execution in DIMM-Based Near-Memory Processing," in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, March 2025, pp. 518–532.
 - [39] B. Y. Cho, Y. Kwon, S. Lym, and M. Erez, "Near Data Acceleration with Concurrent Host Access," in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, May 2020, pp. 818–831.
 - [40] S. Gupta, N. Madan, S. Puthoor, N. Jayasena, and S. Dwarkadas, "Concurrent PIM and Load/Store Servicing in PIM-Enabled Memory," in *2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, May 2025, pp. 320–334.
 - [41] S. Yu, H. Kim, K. Jeun, S. Hwang, S. Cho, and E. Lee, "CompPASS: A Compatible PIM Protocol Architecture and Scheduling Solution for Processor-PIM Collaboration," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25, 2025, p. 49–62.
 - [42] Y.-C. Kwon, S. H. Lee, J. Lee, S.-H. Kwon, J. M. Ryu, J.-P. Son, O. Seongil, H.-S. Yu, H. Lee, S. Y. Kim, Y. Cho, J. G. Kim, J. Choi, H.-S. Shin, J. Kim, B. Phuah, H. Kim, M. J. Song, A. Choi, D. Kim, S. Kim, E.-B. Kim, D. Wang, S. Kang, Y. Ro, S. Seo, J. Song, J. Youn, K. Sohn, and N. S. Kim, "25.4 A 20nm 6GB Function-In-Memory DRAM, Based on HBM2 with a 1.2TFLOPS Programmable Computing Unit Using Bank-Level Parallelism, for Machine Learning Applications," in *2021 IEEE International Solid-State Circuits Conference (ISSCC)*, vol. 64, Feb 2021, pp. 350–352.
 - [43] JEDEC, "JEDEC JESD209-5C: Low Power Double Data Rate 5 (LPDDR5)," JEDEC Solid State Technology Association, Tech. Rep., 6 2023, revision of JESD209-5B (June 2021).
 - [44] Y. Kim, V. Seshadri, D. Lee, J. Liu, and O. Mutlu, "A case for exploiting subarray-level parallelism (SALP) in DRAM," in *2012 39th Annual International Symposium on Computer Architecture (ISCA)*, June 2012, pp. 368–379.
 - [45] D. Lee, Y. Kim, V. Seshadri, J. Liu, L. Subramanian, and O. Mutlu, "Tiered-latency DRAM: A low latency and low cost DRAM architecture," in *2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2013, pp. 615–626.
 - [46] C. Ortega, Y. Falevoz, and R. Ayrignac, "PIM-AI: A Novel Architecture for High-Efficiency LLM Inference," *arXiv e-prints*, p. arXiv:2411.17309, Nov. 2024.
 - [47] M. Barkhordar, A. Tabatabaie, M. Sadrosadati, C. Giannoula, J. G. Luna, I. El Hajj, O. Mutlu, and A. R. Alameldeen, "ALPHA-PIM: Analysis of Linear Algebraic Processing for High-Performance Graph Applications on a Real Processing-In-Memory System," in *2025 IEEE International Symposium on Workload Characterization (IISWC)*, Oct 2025, pp. 257–271.
 - [48] C. Giannoula, P. Yang, I. Fernandez, J. Yang, S. Durvasula, Y. X. Li, M. Sadrosadati, J. G. Luna, O. Mutlu, and G. Pekhimenko, "PyGim: An Efficient Graph Neural Network Library for Real Processing-In-Memory Architectures," *Proc. ACM Meas. Anal. Comput. Syst.*, vol. 8, no. 3, Dec. 2024.
 - [49] C. Giannoula, I. Fernandez, J. Gómez-Luna, N. Koziris, G. Goumas, and O. Mutlu, "SparseP: Efficient Sparse Matrix Vector Multiplication on Real Processing-In-Memory Architectures," in *2022 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, July 2022, pp. 288–291.
 - [50] Y. Kwon, K. Vladimirov, N. Kim, W. Shin, J. Won, M. Lee, H. Joo, H. Choi, G. Kim, B. An, J. Kim, J. Lee, I. Kim, J. Park, C. Park, Y. Song, B. Yang, H. Lee, S. Kim, D. Kwon, S. Lee, K. Kim, S. Oh, J. Park, G. Hong, D. Ka, K. Hwang, J. Park, K. Kang, J. Kim, J. Jeon, M. Lee, M. Shin, M. Shin, J. Cha, C. Jung, K. Chang, C. Jeong, E. Lim, I. Park, J. Chun, and S. Hynix, "System Architecture and Software Stack for GDDR6-AiM," in *2022 IEEE Hot Chips 34 Symposium (HCS)*, Aug 2022, pp. 1–25.
 - [51] S. Lee, S.-h. Kang, J. Lee, H. Kim, E. Lee, S. Seo, H. Yoon, S. Lee, K. Lim, H. Shin, J. Kim, O. Seongil, A. Iyer, D. Wang, K. Sohn, and N. S. Kim, "Hardware Architecture and Software Stack for PIM Based on Commercial DRAM Technology: Industrial Product," in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, June 2021, pp. 43–56.
 - [52] D. Guo, D. Yang, H. Zhang, J. Song, P. Wang, Q. Zhu, R. Xu, R. Zhang, S. Ma, X. Bi, X. Zhang, X. Yu, Y. Wu, Z. F. Wu, Z. Gou, Z. Shao, Z. Li, Z. Gao, A. Liu, B. Xue, B. Wang, B. Wu, B. Feng, C. Lu, C. Zhao, C. Deng, C. Ruan, D. Dai, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Xu, H. Ding, H. Gao, H. Qu, H. Li, J. Guo, J. Li, J. Chen, J. Yuan, J. Tu, J. Qiu, J. Li, J. L. Cai, J. Ni, J. Liang, J. Chen, K. Dong, K. Hu, K. You, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Zhao, L. Wang, L. Zhang, L. Xu, L. Xia, M. Zhang, M. Zhang, M. Tang, M. Zhou, M. Li, M. Wang, M. Li, N. Tian, P. Huang, P. Zhang, Q. Wang, Q. Chen, Q. Du, R. Ge, R. Zhang, R. Pan, R. Wang, R. J. Chen, R. L. Jin, R. Chen, S. Lu, S. Zhou, S. Chen, S. Ye, S. Wang, S. Yu, S. Zhou, S. Pan, S. S. Li, S. Zhou, S. Wu, T. Yun, T. Pei, T. Sun, T. Wang, W. Zeng, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, W. L. Xiao, W. An, X. Liu, X. Wang, X. Chen, X. Nie, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yang, X. Li, X. Su, X. Lin, X. Q. Li, X. Jin, X. Shen, X. Chen, X. Sun, X. Wang, X. Song, X. Zhou, X. Wang, X. Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. Zhang, Y. Xu, Y. Li, Y. Zhao, Y. Sun, Y. Wang, Y. Yu, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Ou, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Xiong, Y. Luo, Y. You, Y. Liu, Y. Zhou, Y. X. Zhu, Y. Huang, Y. Li, Y. Zheng, Y. Zhu, Y. Ma, Y. Tang, Y. Zha, Y. Yan, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Xie, Z. Zhang, Z. Hao, Z. Ma, Z. Yan, Z. Wu, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Pan, Z. Huang, Z. Xu, Z. Zhang, and Z. Zhang, "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025.
 - [53] S. Rixner, W. J. Dally, U. J. Kapasi, P. Mattson, and J. D. Owens, "Memory access scheduling," in *Proceedings of the 27th Annual International Symposium on Computer Architecture*, ser. ISCA '00, 2000, p. 128–138.
 - [54] D. M. Zuvareff and J. I. Robinson, "Controller for a synchronous DRAM that maximizes throughput by allowing memory requests and commands to be issued out of order," May 1997, US Patent 5,630,096.
 - [55] T. Moscibroda and O. Mutlu, "Memory Performance Attacks: Denial of Memory Service in Multi-Core Systems," in *16th USENIX Security Symposium (USENIX Security 07)*, Aug 2007.
 - [56] O. Mutlu and T. Moscibroda, "Stall-Time Fair Memory Access Scheduling for Chip Multiprocessors," in *40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007)*, Dec 2007, pp. 146–160.
 - [57] O. Mutlu and T. Moscibroda, "Parallelism-Aware Batch Scheduling: Enhancing both Performance and Fairness of Shared DRAM Systems," in *2008 International Symposium on Computer Architecture*, June 2008, pp. 63–74.
 - [58] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "The Blacklisting Memory Scheduler: Achieving high performance and fairness at low cost," in *2014 IEEE 32nd International Conference on Computer Design (ICCD)*, Oct 2014, pp. 8–15.
 - [59] Y. Kim, D. Han, O. Mutlu, and M. Harchol-Balter, "ATLAS: A scalable and high-performance scheduling algorithm for multiple memory controllers," in *HPCA - 16 2010 The Sixteenth International Symposium on High-Performance Computer Architecture*, Jan 2010, pp. 1–12.
 - [60] H. Luo, Y. C. Tuğrul, F. N. Bostancı, A. Olgun, A. G. Yağlıkçı, and O. Mutlu, "Ramulator 2.0: A Modern, Modular, and Extensible DRAM Simulator," *IEEE Computer Architecture Letters*, vol. 23, no. 1, pp. 112–116, Jan 2024.
 - [61] CMU-SAFARI, "Ramulator V2.0a," <https://github.com/CMU-SAFARI/ramulator2>, 2023, accessed: 2025-11-05.
 - [62] Y. Kim, W. Yang, and O. Mutlu, "Ramulator: A Fast and Extensible DRAM Simulator," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 45–49, 2016.
 - [63] L. Steiner, T. Psota, M. Mörz, D. Christ, M. Jung, and N. Wehn, "DRAMPower 5: An Open-Source Power Simulator for Current Generation DRAM Standards," in *Proceedings of the Rapid Simulation and Performance Evaluation for Design Workshop*, ser. RAPIDO '25, 2025, p. 8–16.
 - [64] Qualcomm Technologies, Inc., "Snapdragon 888 5G Mobile Platform," <https://www.qualcomm.com/smartphones/products/8-series/snapdragon-888-5g-mobile-platform>, 2020, accessed: 2026-02-19.
 - [65] D. Sanchez and C. Kozyrakis, "ZSim: fast and accurate microarchitectural simulation of thousand-core systems," in *Proceedings of the 40th Annual International Symposium on Computer Architecture*, ser. ISCA '13, 2013, p. 475–486.
 - [66] Xiaomi Corporation, "Xiaomi Mi 11 Pro - Technical Specifications," <https://www.mi.com/mi11pro/specs>, 2021, accessed: 2026-02-19.
 - [67] F. Developers, "Stalker — Frida Documentation," <https://frida.re/docs/stalker/>, 2025, accessed: 2025-11-15.
 - [68] Google LLC, "Android 15," <https://developer.android.google.cn/about/versions/15>, 2024, accessed: 2025-11-05.
 - [69] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to GPU codes," in *2012 Innovative Parallel Computing (InPar)*, May 2012, pp. 1–10.
 - [70] Standard Performance Evaluation Corporation (SPEC), "SPEC CPU2017," <https://www.spec.org/cpu2017/>, 2017, accessed: 2025-11-15.
 - [71] BigScience, "BigScience Language Open-science Open-access Multilingual (BLOOM) Language Model," <https://huggingface.co/bigscience/bloom-1b1>, 2022, accessed: 2025-11-05.
 - [72] A. Yang, B. Yang, B. Hui, B. Zheng, B. Yu, C. Zhou, C. Li, C. Li, D. Liu, F. Huang, G. Dong, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Ma, J. Yang, J. Xu, J. Zhou, J. Bai, J. He, J. Lin, K. Dang, K. Lu, K. Chen, K. Yang, M. Li, M. Xue, N. Ni, P. Zhang, P. Wang, R. Peng, R. Men, R. Gao, R. Lin, S. Wang, S. Bai, S. Tan, T. Zhu, T. Li, T. Liu, W. Ge, X. Deng, X. Zhou, X. Ren, X. Zhang, X. Wei, X. Ren, X. Liu, Y. Fan, Y. Yao, Y. Zhang, Y. Wan, Y. Chu, Y. Liu, Z. Cui, Z. Zhang, Z. Guo, and Z. Fan, "Qwen2 Technical Report," *arXiv e-prints*, p. arXiv:2407.10671, Jul. 2024.
 - [73] C. Shin, J. Song, S. Na, J. Sung, H. Jang, and J. Lee, "FALA: Locality-Aware PIM-Host Cooperation for Graph Processing with Fine-Grained Column Access," in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO '25, 2025, p. 1520–1534.
 - [74] V. Seshadri, D. Lee, T. Mullins, H. Hassan, A. Boroumand, J. Kim, M. A. Kozuch, O. Mutlu, P. B. Gibbons, and T. C. Mowry, "Ambit: In-Memory Accelerator for Bulk

- Bitwise Operations Using Commodity DRAM Technology,” in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2017, pp. 273–287.
- [75] N. Hajinazar, G. F. Oliveira, S. Gregorio, J. A. D. Ferreira, N. M. Ghiasi, M. Patel, M. Alser, S. Ghose, J. Gómez-Luna, and O. Mutlu, “SIMDRAM: a framework for bit-serial SIMD processing using DRAM,” in *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’21, 2021, p. 329–345.
- [76] G. F. Oliveira, A. Olgun, A. G. Yağlıkcı, F. N. Bostancı, J. Gómez-Luna, S. Ghose, and O. Mutlu, “MIMDRAM: An End-to-End Processing-Using-DRAM System for High-Throughput, Energy-Efficient and Programmer-Transparent Multiple-Instruction Multiple-Data Computing,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, March 2024, pp. 186–203.
- [77] G. F. de Oliveira Junior, M. Kabra, Y. Guo, K. Chen, A. G. Yağlıkcı, M. Soysal, M. Sadrosadati, J. Olivares Bueno, S. Ghose, J. Gómez-Luna, and O. Mutlu, “Proteus: Achieving High-Performance Processing-Using-DRAM with Dynamic Bit-Precision, Adaptive Data Representation, and Flexible Arithmetic,” in *Proceedings of the 39th ACM International Conference on Supercomputing*, ser. ICS ’25, 2025, p. 473–494.
- [78] S. Li, D. Niu, K. T. Malladi, H. Zheng, B. Brennan, and Y. Xie, “DRISA: A DRAM-based Reconfigurable In-Situ Accelerator,” in *2017 50th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2017, pp. 288–301.
- [79] S. Aga, S. Jeloka, A. Subramaniyan, S. Narayanasamy, D. Blaauw, and R. Das, “Compute Caches,” in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2017, pp. 481–492.
- [80] Z. Wang, C. Liu, N. Beckmann, and T. Nowatzki, “Affinity Alloc: Taming Not-So Near-Data Computing,” in *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2023, pp. 784–799.
- [81] Z. Wang, C. Liu, A. Arora, L. John, and T. Nowatzki, “Infinity Stream: Portable and Programmer-Friendly In-/Near-Memory Fusion,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, ser. ASPLOS 2023, 2023, p. 359–375.
- [82] Z. Wang, J. Weng, S. Liu, and T. Nowatzki, “Near-Stream Computing: General and Transparent Near-Cache Acceleration,” in *2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, April 2022, pp. 331–345.
- [83] M. Orenes-Vera, E. Tureci, D. Wentzlaff, and M. Martonosi, “Dalorex: A Data-Local Program Execution and Architecture for Memory-bound Applications,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb 2023, pp. 718–730.
- [84] C. Eckert, X. Wang, J. Wang, A. Subramaniyan, R. Iyer, D. Sylvester, D. Blaauw, and R. Das, “Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks,” in *2018 ACM/IEEE 45th Annual International Symposium on Computer Architecture (ISCA)*, 2018, pp. 383–396.
- [85] A. K. Ramanathan, G. S. Kalsi, S. Srinivasa, T. M. Chandran, K. R. Pillai, O. J. Omer, V. Narayanan, and S. Subramoney, “Look-Up Table based Energy Efficient Processing in Cache Support for Neural Network Acceleration,” in *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct 2020, pp. 88–101.
- [86] A. Boroumand, S. Ghose, M. Patel, H. Hassan, B. Lucia, R. Ausavarungnirun, K. Hsieh, N. Hajinazar, K. T. Malladi, H. Zheng, and O. Mutlu, “CoNDA: Efficient Cache Coherence Support for Near-Data Accelerators,” in *2019 ACM/IEEE 46th Annual International Symposium on Computer Architecture (ISCA)*, June 2019, pp. 629–642.
- [87] D. Lee, J. So, M. AHN, J.-G. Lee, J. Kim, J. Cho, R. Oliver, V. C. Thummala, R. S. JV, S. S. Upadhyay, M. I. Khan, and J. H. Kim, “Improving In-Memory Database Operations with Acceleration DIMM (AxDIMM),” in *Proceedings of the 18th International Workshop on Data Management on New Hardware*, ser. DaMoN ’22, 2022.
- [88] C. Li, Y. Yin, X. Wu, J. Zhu, Z. Gao, D. Niu, Q. Wu, X. Si, Y. Xie, C. Zhang, and G. Sun, “H2-LLM: Hardware-Dataflow Co-Exploration for Heterogeneous Hybrid-Bonding-based Low-Batch LLM Inference,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25, 2025, p. 194–210.
- [89] Z. Yue, Y. Wang, C. Li, S. Wei, Y. Hu, and S. Yin, “3D-PATH: A Hierarchy LUT Processing-in-memory Accelerator with Thermal-aware Hybrid Bonding Integration,” in *Proceedings of the 58th IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’25, 2025, p. 78–93.
- [90] J. Ahn, S. Hong, S. Yoo, O. Mutlu, and K. Choi, “A scalable processing-in-memory accelerator for parallel graph processing,” *SIGARCH Comput. Archit. News*, vol. 43, no. 3S, p. 105–117, Jun. 2015.
- [91] J. H. Lee, H. Zhang, V. Lagrange, P. Krishnamoorthy, X. Zhao, and Y. S. Ki, “SmartSSD: FPGA Accelerated Near-Storage Data Analytics on SSD,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 110–113, July 2020.
- [92] K. Chen, R. Nadig, M. Frouzakis, N. M. Ghiasi, Y. Liang, H. Mao, J. Park, M. Sadrosadati, and O. Mutlu, “REIS: A High-Performance and Energy-Efficient Retrieval System with In-Storage Processing,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ser. ISCA ’25, 2025, p. 1171–1192.
- [93] M. Kabra, R. Nadig, H. Gupta, R. Bera, M. Frouzakis, V. Arulchelvan, Y. Liang, H. Mao, M. Sadrosadati, and O. Mutlu, “CIPHERMATCH: Accelerating Homomorphic Encryption-Based String Matching via Memory-Efficient Data Packing and In-Flash Processing,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 2, ser. ASPLOS ’25, 2025, p. 111–130.
- [94] N. M. Ghiasi, M. Sadrosadati, H. Mustafa, A. Gollwitzer, C. Firtina, J. Eudine, H. Mao, J. Lindegger, M. B. Cavlak, M. Alser, J. Park, and O. Mutlu, “MegIS: High-Performance, Energy-Efficient, and Low-Cost Metagenomic Analysis with In-Storage Processing,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, June 2024, pp. 660–677.
- [95] N. Mansouri Ghiasi, J. Park, H. Mustafa, J. Kim, A. Olgun, A. Gollwitzer, D. Senol Cali, C. Firtina, H. Mao, N. Almadhoun Alser, R. Ausavarungnirun, N. Vijaykumar, M. Alser, and O. Mutlu, “GenStore: a high-performance in-storage processing system for genome sequence analysis,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, ser. ASPLOS ’22, 2022, p. 635–654.
- [96] S.-W. Jun, M. Liu, S. Lee, J. Hicks, J. Ankcorn, M. King, S. Xu, and Arvind, “BlueDBM: an appliance for big data analytics,” *SIGARCH Comput. Archit. News*, vol. 43, no. 3S, p. 1–13, Jun. 2015.
- [97] M. Soysal, K. Koliogeorgi, C. Firtina, N. M. Ghiasi, R. Nadig, H. Mao, G. F. de Oliveira Junior, Y. Liang, K. Zambaku, M. Sadrosadati, and O. Mutlu, “MARS: Processing-In-Memory Acceleration of Raw Signal Genome Analysis Inside the Storage Subsystem,” in *Proceedings of the 39th ACM International Conference on Supercomputing*, ser. ICS ’25, 2025, p. 513–534.
- [98] A. Mamdouh, H. Geng, M. Niemier, X. Sharon Hu, and D. Reis, “Shared-PIM: Enabling Concurrent Computation and Data Flow for Faster Processing-in-DRAM,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 11, pp. 4395–4404, Nov 2025.
- [99] V. Seshadri, Y. Kim, C. Fallin, D. Lee, R. Ausavarungnirun, G. Pekhimenko, Y. Luo, O. Mutlu, P. B. Gibbons, M. A. Kozuch, and T. C. Mowry, “RowClone: fast and energy-efficient in-DRAM bulk data copy and initialization,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-46, 2013, p. 185–197.
- [100] A. Boroumand, S. Ghose, G. F. Oliveira, and O. Mutlu, “Polynesia: Enabling High-Performance and Energy-Efficient Hybrid Transactional/Analytical Databases with Hardware/Software Co-Design,” in *2022 IEEE 38th International Conference on Data Engineering (ICDE)*, May 2022, pp. 2997–3011.
- [101] I. E. Y. uksel, Y. C. Tuğrul, A. Olgun, F. N. Bostancı, A. G. Yağlıkcı, G. F. Oliveira, H. Luo, J. Gómez-Luna, M. Sadrosadati, and O. Mutlu, “Functionally-Complete Boolean Logic in Real DRAM Chips: Experimental Characterization and Analysis,” in *2024 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, March 2024, pp. 280–296.
- [102] L. Nai, R. Hadidi, J. Sim, H. Kim, P. Kumar, and H. Kim, “GraphPIM: Enabling Instruction-Level PIM Offloading in Graph Computing Frameworks,” in *2017 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2017, pp. 457–468.
- [103] B. Tian, Q. Chen, and M. Gao, “ABNDP: Co-optimizing Data Access and Load Balance in Near-Data Processing,” in *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Volume 3, ser. ASPLOS 2023, 2023, p. 3–17.
- [104] Z. Zhou, C. Li, F. Yang, and G. Sun, “DIMM-Link: Enabling Efficient Inter-DIMM Communication for Near-Memory Processing,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb 2023, pp. 302–316.
- [105] B. Tian, Y. Li, J. Li, S. Cai, and M. Gao, “NDPBridge: Enabling Cross-Bank Coordination in Near-DRAM-Bank Processing Architectures,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture*, ser. ISCA ’24, 2024, pp. 628–643.
- [106] W. Sun, Z. Li, S. Yin, S. Wei, and L. Liu, “ABC-DIMM: Alleviating the Bottleneck of Communication in DIMM-based Near-Memory Processing with Inter-DIMM Broadcast,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, June 2021, pp. 237–250.
- [107] H. Son, G. Jonatan, X. Wu, H. Cho, K. Shiydikar, J. L. Abellán, A. Joshi, D. Kaeli, and J. Kim, “PIMnet: A Domain-Specific Network for Efficient Collective Communication in Scalable PIM,” in *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, March 2025, pp. 1557–1572.
- [108] D. Fujiki, “MVC: Enabling Fully Coherent Multi-Data-Views through the Memory Hierarchy with Processing in Memory,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO ’23, 2023, p. 800–814.
- [109] A. Boroumand, S. Ghose, M. Patel, H. Hassan, B. Lucia, K. Hsieh, K. T. Malladi, H. Zheng, and O. Mutlu, “LazyPIM: An Efficient Cache Coherence Mechanism for Processing-in-Memory,” *IEEE Computer Architecture Letters*, vol. 16, no. 1, pp. 46–50, Jan 2017.
- [110] Y.-S. Moon, Y. Kwon, H.-S. Kim, D.-g. Kim, H. H. Lee, and K. Park, “The Compact Memory Scheduling Maximizing Row Buffer Locality,” in *Proceedings of the 3rd JILP Workshop on Computer Architecture Competitions: Memory Scheduling Championship (MSC)*, June 2012.
- [111] J. Reineke, I. Liu, H. D. Patel, S. Kim, and E. A. Lee, “PRET DRAM controller: Bank privatization for predictability and temporal isolation,” in *2011 Proceedings of the Ninth IEEE/ACM/IFIP International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*, Oct 2011, pp. 99–108.
- [112] L. Chen, Y. Cao, S. Kabala, and P. Shukla, “Pre-Read and Write-Leak Memory Scheduling Algorithm,” in *Proceedings of the 1st Memory Scheduling Championship (MSC)*. JILP Workshop on Computer Architecture Competitions, June 2012, held in conjunction with ISCA-39.
- [113] C. J. Lee, E. Ebrahimi, V. Narasiman, O. Mutlu, and Y. N. Patt, “DRAM-Aware Last-Level Cache Replacement,” High Performance Systems Group, Department of Electrical and Computer Engineering, The University of Texas at Austin, Austin, TX, Tech. Rep. TR-HPS-2010-007, December 2010.
- [114] Y. Kim, M. Papamichael, O. Mutlu, and M. Harchol-Balter, “Thread Cluster Memory Scheduling: Exploiting Differences in Memory Access Behavior,” in *2010 43rd Annual IEEE/ACM International Symposium on Microarchitecture*, 2010, pp. 65–76.
- [115] W. El-Reedy, A. A. El-Moursy, and H. A. Fahmy, “High Performance Memory Requests Scheduling Technique for Multicore Processors,” in *2012 IEEE 14th International Conference on High Performance Computing and Communication & 2012 IEEE 9th International Conference on Embedded Software and Systems*, June 2012, pp. 127–134.

- [116] A. S. Elhelw, A. El-Moursy, and H. A. H. Fahmy, "Adaptive Time-Based Least Memory Intensive Scheduling," in *2015 IEEE 9th International Symposium on Embedded Multicore/Many-core Systems-on-Chip*, Sep. 2015, pp. 167–174.
- [117] K. J. Nesbit, N. Aggarwal, J. Laudon, and J. E. Smith, "Fair Queuing Memory Systems," in *2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06)*, Dec 2006, pp. 208–222.
- [118] B. Akesson and K. Goossens, "Architectures and modeling of predictable memory controllers for improved system integration," in *2011 Design, Automation & Test in Europe*, March 2011, pp. 1–6.
- [119] W. Liu, P. Huang, T. Kun, T. Lu, K. Zhou, C. Li, and X. He, "LAMS: A latency-aware memory scheduling policy for modern DRAM systems," in *2016 IEEE 35th International Performance Computing and Communications Conference (IPCCC)*, Dec 2016, pp. 1–8.
- [120] L. Subramanian, D. Lee, V. Seshadri, H. Rastogi, and O. Mutlu, "BLISS: Balancing Performance, Fairness and Complexity in Memory Access Scheduling," *IEEE Transactions on Parallel and Distributed Systems*, vol. 27, no. 10, pp. 3071–3087, Oct 2016.
- [121] J. Zhao, O. Mutlu, and Y. Xie, "FIRM: Fair and High-Performance Memory Control for Persistent Memory Systems," in *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, 2014, pp. 153–165.
- [122] R. Ausavarungnirun, S. Ghose, O. Kayiran, G. H. Loh, C. R. Das, M. T. Kandemir, and O. Mutlu, "Exploiting Inter-Warp Heterogeneity to Improve GPGPU Performance," in *2015 International Conference on Parallel Architecture and Compilation (PACT)*, Oct 2015, pp. 25–38.
- [123] H. Shah, A. Raabe, and A. Knoll, "Bounding WCET of applications using SDRAM with Priority Based Budget Scheduling in MPSoCs," in *2012 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, March 2012, pp. 665–670.
- [124] M. Paolieri, E. Quiñones, and F. J. Cazorla, "Timing effects of DDR memory systems in hard real-time multicore architectures: Issues and solutions," *ACM Trans. Embed. Comput. Syst.*, vol. 12, no. 1s, Mar. 2013.
- [125] H. Usui, L. Subramanian, K. K.-W. Chang, and O. Mutlu, "Dash: Deadline-aware high-performance memory scheduler for heterogeneous systems with hardware accelerators," *ACM Trans. Archit. Code Optim.*, vol. 12, no. 4, Jan. 2016.
- [126] J. Fang, M. Wang, and Z. Wei, "A memory scheduling strategy for eliminating memory access interference in heterogeneous system," *J. Supercomput.*, vol. 76, no. 4, p. 3129–3154, Apr. 2020.
- [127] D. Li and T. M. Aamodt, "Inter-Core Locality Aware Memory Scheduling," *IEEE Computer Architecture Letters*, vol. 15, no. 1, pp. 25–28, Jan 2016.
- [128] A. Jog, O. Kayiran, A. Pattnaik, M. T. Kandemir, O. Mutlu, R. Iyer, and C. R. Das, "Exploiting Core Criticality for Enhanced GPU Performance," in *Proceedings of the 2016 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Science*, ser. SIGMETRICS '16, 2016, p. 351–363.
- [129] R. Ausavarungnirun, K. K.-W. Chang, L. Subramanian, G. H. Loh, and O. Mutlu, "Staged memory scheduling: achieving high performance and scalability in heterogeneous systems," in *Proceedings of the 39th Annual International Symposium on Computer Architecture*, ser. ISCA '12, 2012, p. 416–427.
- [130] O. Kayiran, N. C. Nachiappan, A. Jog, R. Ausavarungnirun, M. T. Kandemir, G. H. Loh, O. Mutlu, and C. R. Das, "Managing GPU Concurrency in Heterogeneous Architectures," in *Proceedings of the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, ser. MICRO-47, 2014, p. 114–126.
- [131] E. Ipek, O. Mutlu, J. F. Martinez, and R. Caruana, "Self-Optimizing Memory Controllers: A Reinforcement Learning Approach," in *2008 International Symposium on Computer Architecture*, June 2008, pp. 39–50.
- [132] O. H. Elgawi, C. S. Leung, M. Lee, and J. H. Chan, "RL-Based Memory Controller for Scalable Autonomous Systems," in *Neural Information Processing Systems*, C. S. Leung, M. Lee, and J. H. Chan, Eds., 2009, pp. 83–92.
- [133] A. Ray and H. Ray, "Proposing ϵ -greedy Reinforcement Learning Technique to Self-Optimize Memory Controllers," in *2021 2nd International Conference on Secure Cyber Computing and Communications (ICSCCC)*, May 2021, pp. 318–323.